

UNIVERZITA PARDUBICE

FAKULTA ELEKTROTECHNIKY A
INFORMATIKY

BAKALÁŘSKÁ PRÁCE

2026

Egor Razboinikov

Univerzita Pardubice

Fakulta Elektrotechniky a Informatiky

Digitalizace evidence pracovní doby

Bakalářská práce

2026

Egor Razboinikov

ZADÁNÍ BAKALÁŘSKÉ PRÁCE

(projektu, uměleckého díla, uměleckého výkonu)

Jméno a příjmení: Egor Razboinikov

Osobní číslo: I21219

Studijní program: B0688A140009 Informační technologie

Téma práce: Digitalizace evidence pracovní doby Zadávající
katedra: Katedra informačních technologií

Zásady pro vypracování

Dokument evidence pracovní doby vyučujícího vyžaduje provázanost rozvrhu s úvazkem vyučujícího při respektování času přestávek. Cílem bakalářské práce je navrhnout, vytvořit, odzkoušet a předat funkční multiplatformní aplikaci jejíž vstupy jsou: rozvrh vyučujícího, čas na oběd, plán dovolených, státní svátky, školní volna a další. Požadovaným výstupem je PDF dokument (tabulka) se všemi náležitostmi potřebnými pro odevzdání. Grafické rozhraní poskytne pohodlné zadávání, umožní dodatečné ruční modifikace a zobrazování náhledu před vygenerováním PDF dokumentu.

Rozsah pracovní zprávy: **min. 30 stran**
Rozsah grafických prací:
Forma zpracování bakalářské práce: **tištěná**

Seznam doporučené literatury:

PŠENČÍKOVÁ, J. Algoritmizace. Kralice na Hané, Computer Media, 2009.
MAREŠ, Martin a VALLA, Tomáš. Průvodce labyrintem algoritmů 2. vydání. Praha: CZ.NIC, 2022. ISBN 978-80-88168-63-8
PECINOVSKÝ, Rudolf. OOP: naučte se myslet a programovat objektivě. Brno: Computer Press, 2010. ISBN 978-80-251-2126-9.

Vedoucí bakalářské práce: **Ing. Radek Matoušek**
Katedra informačních technologií

Datum zadání bakalářské práce: **15. prosince 2024**
Termín odevzdání bakalářské práce: **16. května 2025**

L.S.

prof. Ing. Petr Doležel, Ph.D. v.r.
děkan

Ing. Jan Panuš, Ph.D. v.r.
vedoucí katedry

V Pardubicích dne 28. února 2025

Prohlašuji:

Práci s názvem Digitalizace evidence pracovní doby jsem vypracoval samostatně. Veškeré literární prameny a informace, které jsem v práci využil, jsou uvedeny v seznamu použité literatury.

Byl jsem seznámen s tím, že se na moji práci vztahují práva a povinnosti vyplývající ze zákona č. 121/2000 Sb., o právu autorském, o právech souvisejících s právem autorským a o změně některých zákonů (autorský zákon), ve znění pozdějších předpisů, zejména se skutečností, že Univerzita Pardubice má právo na uzavření licenční smlouvy o užití této práce jako školního díla podle § 60 odst. 1 autorského zákona, a s tím, že pokud dojde k užití této práce mnou nebo bude poskytnuta licence o užití jinému subjektu, je Univerzita Pardubice oprávněna ode mne požadovat přiměřený příspěvek na úhradu nákladů, které na vytvoření díla vynaložila, a to podle okolností až do jejich skutečné výše.

Beru na vědomí, že v souladu s § 47b zákona č. 111/1998 Sb., o vysokých školách a o změně a doplnění dalších zákonů (zákon o vysokých školách), ve znění pozdějších předpisů, a směrnicí Univerzity Pardubice č. 7/2019 Pravidla pro odevzdávání, zveřejňování a formální úpravu závěrečných prací, ve znění pozdějších dodatků, bude práce zveřejněna prostřednictvím Digitální knihovny Univerzity Pardubice.

V Pardubicích dne 9. 5. 2026

Egor Razboinikov v.r.

PODĚKOVÁNÍ

Tady chtěl bych poděkovat svému vedoucímu Ing. Radek Matoušek za hezké vedení bakalářské práce a rychle odpovědi. Také rád bych poděkoval svým rodičům a kamarádům za jejich podporu po celou dobu studia.

ANOTACE

Cílem této bakalářské práce je navrhnout, implementovat a ověřit multiplatformní desktopovou aplikaci pro digitalizaci evidence pracovní doby vyučujících. Aplikace automaticky zpracuje data z rozvrhu (STAG), zohlední polední přestávky, plánované nepřítomnosti, státní svátky a pravidla EPD a z nich vytvoří přehledný výkaz. Výstupem je PDF v podobě odpovídající požadavkům fakulty, přičemž vstupní údaje lze pohodlně upravit v intuitivním grafickém rozhraní. Práce popisuje použitou metodiku, volbu technologií a architektury a shrnuje výsledky praktického ověření a testování.

KLÍČOVÁ SLOVA

C#, .NET, Avalonia UI, MVVM, AXAML, ReactiveUI, SQLite, Entity Framework Core, QuestPDF, desktopová aplikace, rozvrh, evidence pracovní doby, generování PDF, automatizace.

TITLE

Digitization of Teachers' Working Time Records

ANNOTATION

The aim of this bachelor thesis is to design, implement, and validate a cross-platform desktop application for digitizing university teachers' working time records. The application automatically processes timetable data (STAG), considers lunch breaks, absences, public holidays, and EPD rules, and produces a clear monthly report. The output is a PDF document compliant with faculty requirements, while the underlying data can be conveniently edited through an intuitive GUI. The thesis presents the methodology, technology and architecture choices, and summarizes the results of practical evaluation and testing.

KEYWORDS

C#, .NET, Avalonia UI, MVVM, AXAML, ReactiveUI, SQLite, Entity Framework Core, QuestPDF, desktop application, timetable, working time records, PDF generation, automation.

OBSAH

SEZNAM ILUSTRACÍ.....10

SEZNAM ZKRATEK A ZNAČEK	10
TERMINOLOGIE	12
ÚVOD.....	13
1 TEORETICKÁ ČÁST	14
1.1 Evidence pracovní doby	14
1.2 Analýza současných přístupů k evidenci pracovní doby	15
1.3 Požadavky na softwarové řešení.....	16
1.3.1 Funkční požadavky	17
1.3.2 Nefunkční požadavky	17
1.3.2 Nefunkční požadavky	18
1.4 Softwarová architektura a návrhové vzory	18
1.4.1 Softwarová architektura	18
1.4.2 Návrhový vzor MVVM	19
1.4.3 Srovnání MVVM a MVC	20
1.4.4 Monolitická, modulární a mikroslužbová architektura.....	20
1.4.5 Zdůvodnění zvoleného řešení	21
1.5 Použité technologie.....	22
1.5.1 Platforma .NET a jazyk C#.....	22
1.5.2 Databázová vrstva: SQLite a jazyk SQL	22
1.5.3 Framework Avalonia UI.....	23
1.5.4 Podpůrné knihovny	23
1.5.5 Shrnutí použitých technologií.....	24
1.6 Vývojové prostředí a podpůrné nástroje.....	24
1.6.1 Microsoft Visual Studio	25
1.6.2 SQLiteStudio	25
1.6.3 Shrnutí vývojového prostředí a podpůrných nástrojů.....	25
2 PRAKTICKÁ ČÁST	27

2.1 Cíl a návrh aplikace	27
2.2 Struktura aplikace	27
2.3 Souborová struktura aplikace	28
2.3.1 Složka Behaviors	29
2.3.2 Složka Controls.....	29
2.3.3 Složka Converters.....	29
2.3.4 Složka Data.....	29
2.3.5 Složka Helpers	30
2.3.6 Složka Messages.....	30
2.3.7 Složka Models	31
2.3.8 Složka Services.....	31
2.3.9 Složka ViewModels	31
2.3.10 Složka Views	31
2.3.11 Složka Kořenové soubory projektu	31
2.4 Struktura databáze	31
2.4.1 Přehled tabulek	33
2.4.2 Shrnutí databázového návrhu	37
2.5 Uživatelské rozhraní aplikace.....	38
2.5.1 Hlavní stránka aplikace	38
2.5.2 Dialogové okno pro vytvoření události	40
2.5.3 Dialogové okno pro změnu události.....	41
2.5.4 Vizuální prezentace pro den.....	42
2.5.5 Vizuální prezentace pro týden	43
2.5.6 Vizuální prezentace pro měsíc	44
2.5.7 Globální nastavení	45
2.5.8 Ukázka PDF.....	46
2.5.9 Shrnutí kapitoly	47

2.6 Algoritmy	47
2.6.1 Algoritmus vyrovnávání hodin	48
2.6.2 Algoritmus importu rozvrhu ze STAG	51
2.7 Testování aplikace	53
2.7.1 Cíl a kritéria přijetí	54
2.7.2 Testovací prostředí	54
2.7.3 Metodika testování	54
2.7.4 Předpokládané rozdíly mezi platformami	55
2.7.5 Testovací scénáře	55
2.7.6 Souhrnná tabulka testů	56
2.7.7 Shrnutí testování a známá omezení	60
ZÁVĚR	61
POUŽITÁ LITERATURA	62

SEZNAM ILUSTRACÍ

Obrázek 1 Tabulka EPD.....	15
Obrázek 2 Architektura MVVM	19
Obrázek 3 Architektura MVC	20
Obrázek 4 Souborová struktura aplikace	29
Obrázek 5 Ukázka kódu třídy AppDbContext	30
Obrázek 6 Relační schéma databáze	33
Obrázek 7 Struktura tabulky Employees	34
Obrázek 8 Struktura tabulky Events	35
Obrázek 9 Struktura tabulky DaySettings.....	36
Obrázek 10 Struktura tabulky SemesterSettings.....	36
Obrázek 11 Struktura tabulky WeekdaySettings.....	37
Obrázek 12 Struktura tabulky ImportBatches.....	37
Obrázek 13 Levý panel aplikace	39
Obrázek 14 Pravý panel aplikace.....	40
Obrázek 15 Dialogové okno pro vytvoření události	41
Obrázek 16 Dialogové okno pro změnu události.....	42
Obrázek 17 Vizuální prezentace pro den	43
Obrázek 18 Vizuální prezentace pro týden	44
Obrázek 19 Vizuální prezentace pro měsíc	45
Obrázek 20 Globální nastavení	46
Obrázek 21 Ukázka PDF	47
Obrázek 22 Algoritmus vyrovnávání hodin	51
Obrázek 23 Algoritmus vyrovnávání hodin	53

SEZNAM ZKRATEK A ZNAČEK

EPD – evidence pracovní doby

FPD – fond pracovní doby

D – dovolená

PC – pracovní cesta

N – nemoc

OČR – ošetřování člena rodiny

L – lékař

S – státní svátek

CSV – Comma-Separated Values

PDF – Portable Document Format

GUI – Graphical User Interface

UI – User Interface

STAG – Studijní agenda

OS – operační systém

DB – databáze

EF Core – Entity Framework Core

FK – Foreign Key

DI – Dependency Injection

IoC – Inversion of Control

ORM – Object–Relational Mapping

CLR – Common Language Runtime

XAML – eXtensible Application Markup Language

AXAML – Avalonia eXtensible Application Markup Language MVC – Model–View–Controller

MVVM – Model–View–ViewModel

JIT – Just-In-Time

CI/CD – Continuous Integration / Continuous Deployment

TERMINOLOGIE

Avalonia UI – Multiplatformní framework pro tvorbu uživatelských rozhraní v .NET s deklarativním popisem obrazovek v AXAML. Umožňuje jednotný vzhled a chování na Windows, Linuxu i macOS.

Framework – Sada knihoven, nástrojů a doporučených postupů, která dává aplikaci strukturu a opakovaně použitelné stavební bloky.

Relační databáze – Datový model založený na tabulkách a vztazích (vazby 1:1, 1:N, N:M) s důrazem na integritu a efektivní dotazování.

MVVM (Model–View–ViewModel) – Vzor, který odděluje data a logiku (Model), prezentační logiku (ViewModel) a zobrazení (View). Zlepšuje testovatelnost a udržitelnost UI.

Uživatelské rozhraní (UI) – Vizuální a interaktivní část aplikace (okna, prvky, šablony), v této práci vytvořená v Avalonia UI/AXAML.

IDE – Integrované vývojové prostředí (editor, debugger, správce závislostí a verzí) pro efektivní vývoj.

VCS – Systémy pro správu verzí (např. Git) pro sledování změn a týmovou spolupráci.

STAG – Studijní agenda, zdroj rozvrhových dat pro import.

EPD – Evidence pracovní doby, oficiální výkaz s měsíčním a týdenním součtem dle interních pravidel.

.

ÚVOD

Evidence pracovní doby představuje důležitou součást personální a administrativní agendy v různých typech organizací. Slouží k zaznamenávání odpracovaného času zaměstnanců, kontrole plnění pracovních povinností a jako podklad pro další související procesy. Správné vedení evidence pracovní doby je významné nejen z hlediska pracovněprávních požadavků, ale také z hlediska přehlednosti, správnosti a efektivity administrativního zpracování údajů.

V praxi může být vedení evidence pracovní doby spojeno s řadou obtíží. Ty se týkají zejména ručního zadávání údajů, nejednotnosti používaných nástrojů, zvýšené chybovosti a časové náročnosti při zpracování nebo kontrole záznamů. Tyto problémy se mohou dále prohlubovat v prostředí, kde je pracovní režim méně pravidelný a kde je třeba zohlednit větší množství vstupních údajů, změn a výjimek.

Cílem této bakalářské práce je navrhnout, implementovat a ověřit softwarové řešení, které usnadní evidenci pracovní doby a přispěje ke zpřehlednění celého procesu. Navržená aplikace má umožnit práci se záznamy pracovní doby, jejich úpravu, zpracování vstupních údajů a vytvoření přehledného výstupu pro další administrativní využití.

Teoretická část práce se nejprve zabývá vymezením evidence pracovní doby a analýzou současných přístupů k jejímu vedení. Dále jsou formulovány požadavky na navržené softwarové řešení a jsou popsány architektonické a technologické prostředky vhodné pro jeho realizaci.

Praktická část práce je věnována návrhu, implementaci a ověření vytvořené aplikace. Zaměřuje se na strukturu aplikace, databázový model, uživatelské rozhraní, algoritmy pro zpracování dat a testování výsledného řešení. Přínosem práce je návrh a realizace aplikace, která může přispět ke zjednodušení evidence pracovní doby a ke snížení administrativní náročnosti tohoto procesu.

.

1 TEORETICKÁ ČÁST

1.1 Evidence pracovní doby

Evidence pracovní doby představuje důležitý nástroj pro zaznamenávání a kontrolu pracovní činnosti zaměstnanců. Jejím hlavním účelem je zajistit přehled o odpracovaném čase, sledovat dodržování pracovní doby a poskytovat podklady pro další administrativní a mzdové procesy. Povinnost vést evidenci pracovní doby je stanovena pracovněprávními předpisy a vztahuje se na všechny zaměstnavatele.

Základem evidence pracovní doby je zaznamenávání začátku a konce pracovní doby, případně dalších skutečností, které ovlivňují její průběh. Součástí evidence jsou obvykle také údaje o přestávkách, odpracovaném čase a různých formách nepřítomnosti zaměstnance. Evidence tak umožňuje rozlišit mezi skutečně odpracovaným časem a dobou, která nebyla odpracována, ale je z právního hlediska uznána.

Důležitým pojmem v této oblasti je fond pracovní doby, který vyjadřuje množství času, které má zaměstnanec v daném období odpracovat. Tento fond je obvykle stanoven na týdenní bázi a může se lišit v závislosti na typu pracovního poměru. V průběhu sledovaného období dochází k porovnávání skutečně odpracovaného času s tímto fondem, přičemž mohou vznikat odchylky, například ve formě přesčasové práce.

V prostředí vysokých škol má evidence pracovní doby svá specifika. Pracovní činnost vyučujících není vždy pravidelná a je často ovlivněna rozvrhem výuky, administrativní činností a dalšími aktivitami. Tato variabilita může ztěžovat přesné a jednoznačné zaznamenávání pracovní doby.

Správně vedená evidence pracovní doby by měla být přehledná, srozumitelná a spolehlivá. Současně by měla umožňovat snadnou kontrolu zaznamenaných údajů a omezovat riziko chyb při jejich zpracování. Tyto požadavky mají význam nejen z hlediska pracovněprávní evidence, ale také z hlediska efektivního administrativního zajištění pracovního procesu.

Den	Začátek pracovní doby	1. přestávka		2. přestávka		Konec pracovní doby	Odpracováno	Přesčas	Neodprac.	Poznámka)
		Od	Do	Od	Do					
1.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
2.										
3.										
4.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
5.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
6.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
7.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
8.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
9.										
10.										
11.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
12.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
13.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
14.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
15.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
16.										
17.										
18.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
19.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
20.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
21.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
22.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
23.										
24.										
25.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
26.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
27.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
28.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
29.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
30.										
31.										
Celkem							168:00:00	00:00:00	00:00:00	

kontr. č. Σ odprac. - přesčas + neodprac. hodin (odpovídá FPD): 168:00:00

Obrázek 1 Tabulka EPD

1.2 Analýza současných přístupů k evidenci pracovní doby

V návaznosti na vymezení evidence pracovní doby jako důležité součásti pracovněprávní a administrativní agendy je vhodné zaměřit pozornost také na způsoby, jakými je tato evidence v praxi vedena. Přestože základní cíl evidence pracovní doby zůstává stejný, konkrétní formy jejího zpracování se mohou mezi jednotlivými organizacemi výrazně lišit. Rozdíly se týkají zejména míry digitalizace, způsobu zaznamenávání údajů i následného vyhodnocování pracovního času.

V praxi se lze setkat s ručním vedením evidence pracovní doby, například v papírové podobě nebo prostřednictvím jednoduchých formulářů. Výhodou tohoto přístupu je jeho technická nenáročnost a snadná dostupnost. Na druhou stranu však ruční evidence zvyšuje riziko chyb při zapisování, přepisování i následné kontrole údajů. Nevýhodou bývá rovněž nižší přehlednost, obtížnější archivace a omezená možnost dalšího zpracování zaznamenaných dat.

Dalším častým přístupem je vedení evidence pracovní doby pomocí tabulkových nástrojů. Tyto nástroje umožňují vytvářet vlastní formuláře, provádět základní výpočty a generovat souhrnné přehledy. Jejich výhodou je určitá flexibilita a relativně snadná dostupnost. Současně však zůstávají do značné míry závislé na ručním zadávání údajů a na správném nastavení použitých vzorců. V případě složitějších situací, jako jsou změny pracovního režimu, nepravidelné rozvržení pracovní doby nebo evidence různých forem nepřítomnosti, může být tento způsob méně přehledný a časově náročný. Problémem může být také nejednotnost používaných šablon a rozdílný způsob jejich vyplňování.

Vyspělejší možnost představují specializované informační systémy určené pro evidenci pracovní doby. Tyto systémy zpravidla umožňují centralizované ukládání dat, automatizaci vybraných úkonů a propojení s dalšími personálními nebo mzdovými procesy. Jejich přínosem bývá vyšší míra kontroly, lepší dostupnost údajů a snížení administrativní zátěže. Nevýhodou však může být vyšší technická a organizační náročnost, omezené možnosti přizpůsobení konkrétním požadavkům uživatele nebo závislost na určitém softwarovém prostředí. V některých případech mohou být tato řešení příliš obecná a nemusí plně odpovídat specifickým potřebám konkrétní pracovní činnosti.

Z hlediska porovnání jednotlivých přístupů je zřejmé, že se liší především mírou automatizace, přehledností a odolností vůči chybám. Ruční a tabulkové způsoby evidence jsou sice snadno dostupné, avšak při pravidelném používání mohou zvyšovat časovou náročnost a riziko nepřesností. Naopak specializované systémy nabízejí vyšší úroveň automatizace, ale nemusí vždy poskytovat dostatečnou jednoduchost, flexibilitu nebo přizpůsobení konkrétním podmínkám.

Tyto skutečnosti jsou zvláště významné v prostředí, kde je pracovní režim proměnlivý a kde je třeba zohledňovat více vstupních faktorů, například plánovanou pracovní činnost, změny v rozvržení práce nebo různé typy nepřítomnosti. V takových podmínkách roste význam řešení, které dokáže spojit přehledné uživatelské rozhraní, jednoduchou práci s daty a automatizované zpracování vybraných operací.

Z uvedené analýzy vyplývá, že vhodné řešení evidence pracovní doby by mělo být přehledné, srozumitelné, snadno použitelné a zároveň dostatečně flexibilní. Současně by mělo omezovat riziko chyb a snižovat administrativní náročnost celého procesu. Tyto požadavky představují důležité východisko pro návrh vlastního softwarového řešení, kterému se věnuje praktická část této práce.

1.3 Požadavky na softwarové řešení

Na základě vymezení evidence pracovní doby a analýzy současných přístupů lze formulovat požadavky, které by mělo splňovat navrhované softwarové řešení. Tyto požadavky vycházejí z potřeby zjednodušit práci s evidencí pracovní doby, zvýšit přehlednost zpracovávaných údajů a omezit riziko chyb, které mohou vznikat při ručním nebo nedostatečně automatizovaném zpracování. Pro účely této práce jsou požadavky rozděleny na funkční a nefunkční.

1.3.1 Funkční požadavky

Funkční požadavky vymezují, jaké činnosti má navržený systém umožňovat a jaké úlohy má plnit. Základním požadavkem je práce se záznamy pracovní doby. Systém by měl umožňovat evidenci údajů souvisejících s pracovním dnem, tedy zejména záznam začátku a konce pracovní doby, přestávek a dalších skutečností, které ovlivňují výslednou podobu evidence.

Dalším požadavkem je možnost úprav jednotlivých údajů. Vzhledem k tomu, že evidence pracovní doby může být ovlivněna různými změnami nebo specifickými situacemi, musí systém umožňovat dodatečnou úpravu záznamů bez nutnosti vytvářet celý výkaz znovu. Tato vlastnost je důležitá zejména v případech, kdy je třeba reagovat na změny v průběhu pracovního dne nebo na doplnění chybějících údajů.

Významnou součástí funkčních požadavků je také zpracování vstupních dat. Navržené řešení by mělo umožňovat načtení a další práci s údaji, které tvoří základ evidence pracovní doby. Důležité je, aby systém dokázal tato data převzít, zpracovat a připravit je pro další úpravy nebo vyhodnocení. Tím se snižuje potřeba opakovaného ručního přepisování údajů a zvyšuje se efektivita celého procesu.

Posledním podstatným funkčním požadavkem je generování výstupů. Výsledkem práce systému by neměly být pouze interně uložené údaje, ale také přehledný a použitelný výstup, který bude možné dále využít pro administrativní účely. Navržené řešení by proto mělo podporovat vytvoření výstupu ve srozumitelné a standardizované podobě.

1.3.2 Nefunkční požadavky

Vedle funkčních požadavků je nutné stanovit také požadavky nefunkční, které určují kvalitativní vlastnosti navrhovaného systému. Jedním z nejdůležitějších nefunkčních požadavků je přehlednost. Práce s evidencí pracovní doby zahrnuje údaje, které musí být snadno čitelné, jednoznačně uspořádané a jednoduše kontrolovatelné. Uživatel by měl mít možnost rychle se orientovat v jednotlivých záznamech a bez obtíží porozumět celkové struktuře evidence.

Dalším významným požadavkem je spolehlivost. Systém musí zpracovávat data konzistentně a poskytovat správné výsledky i při opakovaném použití. Spolehlivost je důležitá zejména proto, že evidence pracovní doby slouží jako podklad pro další administrativní nebo kontrolní činnosti, a proto je nezbytné, aby výstupy systému odpovídaly zadaným údajům a byly ověřitelné.

Důležitou vlastností navrženého řešení je také snadná použitelnost. Cílem není vytvořit technicky složitý systém určený pouze pro zkušené uživatele, ale nástroj, který bude možné

používat v běžné praxi bez zbytečně vysokých nároků na technické znalosti. Uživatelské prostředí by proto mělo být intuitivní a jednotlivé operace by měly odpovídat očekávanému způsobu práce s evidencí.

Mezi zásadní nefunkční požadavky patří rovněž omezení chybovosti. V běžné praxi mohou chyby vznikat při ručním zapisování, přepisování nebo vyhodnocování údajů. Navržený systém by měl tato rizika snižovat, a to především podporou automatizace vybraných operací, jednoznačnou strukturou dat a přehledným způsobem jejich zobrazení.

1.3.2 Nefunkční požadavky

Z formulovaných požadavků vyplývá, že navržené softwarové řešení by mělo spojovat funkční podporu práce se záznamy pracovní doby s kvalitativními vlastnostmi, které zajistí jeho praktickou využitelnost. Systém by měl umožňovat zpracování vstupních dat, úpravu jednotlivých záznamů a generování přehledných výstupů. Současně by měl být přehledný, spolehlivý, snadno použitelný a měl by omezovat riziko vzniku chyb.

Takto stanovené požadavky představují důležité východisko pro návrh architektury a volbu vhodných technologických prostředků. Následující kapitola se proto zaměřuje na softwarovou architekturu a návrhové přístupy, které jsou pro realizaci navrženého řešení relevantní.

1.4 Softwarová architektura a návrhové vzory

Na základě požadavků formulovaných v předchozí kapitole je třeba stanovit takový architektonický přístup, který umožní navrhnout přehledné, udržovatelné a spolehlivé softwarové řešení. Volba vhodné softwarové architektury ovlivňuje nejen vnitřní strukturu aplikace, ale také způsob zpracování dat, oddělení odpovědností mezi jednotlivými částmi systému a možnosti jeho dalšího rozvoje. V případě aplikace zaměřené na evidenci pracovní doby je důležité zejména to, aby architektura podporovala přehlednou práci s daty, snadné úpravy jednotlivých částí systému a jednoznačné oddělení uživatelského rozhraní od aplikační logiky.

1.4.1 Softwarová architektura

Softwarovou architekturu lze chápat jako soubor základních rozhodnutí o struktuře systému, jeho hlavních komponentách a vztazích mezi nimi. Určuje, z jakých částí se aplikace skládá, jaké odpovědnosti jednotlivé části nesou a jakým způsobem spolu komunikují. Správně navržená architektura přispívá ke srozumitelnosti systému, usnadňuje jeho údržbu a umožňuje efektivnější rozšiřování funkcionality v budoucnu.

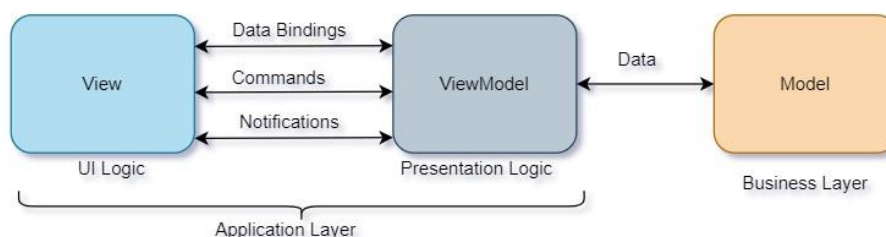
V kontextu této práce má volba architektury význam především proto, že navržené řešení pracuje s více typy činností, například se zpracováním vstupních dat, jejich úpravou, vyhodnocením a tvorbou výstupů. Pokud by tyto části nebyly dostatečně odděleny, mohlo by docházet ke zhoršení přehlednosti kódu, vyšší provázanosti jednotlivých komponent a obtížnější testovatelnosti aplikace. Z tohoto důvodu je vhodné zvolit takový architektonický přístup, který umožní rozdělit systém na logicky oddělené části a zároveň zachovat jednoduchost celého řešení.

1.4.2 Návrhový vzor MVVM

Jedním z vhodných návrhových vzorů pro aplikace s grafickým uživatelským rozhraním je MVVM, tedy Model–View–ViewModel. Tento přístup rozděluje aplikaci do tří základních částí. Model představuje data a aplikační logiku, View zajišťuje zobrazení uživatelského rozhraní a ViewModel tvoří prostřední vrstvu mezi daty a zobrazením. Úkolem ViewModelu je zprostředkovávat komunikaci mezi uživatelským rozhraním a datovou vrstvou a poskytovat takové struktury a operace, které lze přímo využít při prezentaci dat.

Význam MVVM spočívá především v oddělení prezentační vrstvy od aplikační logiky. Díky tomu lze uživatelské rozhraní navrhovat a upravovat odděleně od zpracování dat a jednotlivé části systému se nestávají zbytečně provázanými. Tento přístup zároveň podporuje přehlednější organizaci kódu a usnadňuje testování, protože logika aplikace není pevně svázána s konkrétní podobou uživatelského rozhraní.

Pro desktopové aplikace využívající deklarativní způsob tvorby rozhraní je MVVM považován za přirozený a často používaný přístup. Umožňuje efektivně pracovat se stavem aplikace, vazbami mezi daty a uživatelským rozhraním a s reakcemi na uživatelské akce. Z hlediska požadavků formulovaných v předchozí kapitole je tento vzor vhodný zejména proto, že podporuje přehlednost, udržitelnost a jednoznačné rozdělení odpovědností.



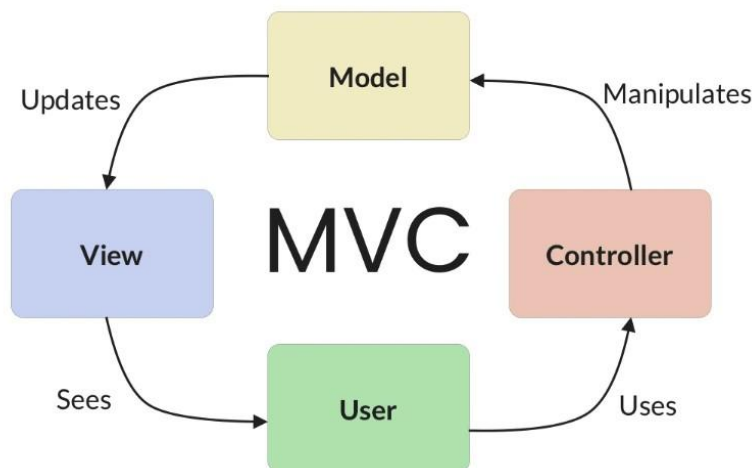
Obrázek 2 Architektura MVVM

1.4.3 Srovnání MVVM a MVC

Vedle vzoru MVVM se v softwarovém inženýrství často používá také návrhový vzor MVC, tedy Model–View–Controller. Stejně jako MVVM i tento přístup usiluje o oddělení jednotlivých odpovědností v aplikaci. Model představuje data a logiku, View slouží k jejich zobrazení a Controller zajišťuje zpracování vstupů od uživatele a řízení toku aplikace.

MVC je tradičně spojován zejména s aplikacemi, ve kterých Controller přímo reaguje na akce uživatele a rozhoduje o tom, jaká data budou zobrazena. Tento přístup je vhodný především v prostředích, kde je potřeba explicitně řídit komunikaci mezi uživatelským vstupem a výstupní vrstvou. V případě moderních desktopových aplikací s deklarativním uživatelským rozhraním však bývá vhodnější MVVM, protože lépe odpovídá principu datových vazeb a oddělení prezentační logiky od samotného zobrazení.

Zatímco MVC kladе důraz na řídicí roli Controlleru, MVVM více využívá zprostředkující vrstvu ViewModelu, která připravuje data pro prezentaci a omezuje přímou závislost mezi rozhraním a logikou aplikace. Pro aplikaci zaměřenou na evidenci pracovní doby je tento přístup vhodnější zejména proto, že podporuje přehlednější práci se stavem formulářů, záznamů a výstupních údajů.



Obrázek 3 Architektura MVC

1.4.4 Monolitická, modulární a mikroslužbová architektura

Při návrhu softwarového řešení je kromě volby návrhového vzoru důležité také zvolit celkové uspořádání aplikace. Jednou z možností je monolitická architektura, při níž je systém vytvářen jako jeden celek. Výhodou takového přístupu je jednoduchost vývoje, nasazení i správy, protože aplikace tvoří jedinou spustitelnou jednotku. Nevýhodou však může být menší přehlednost při růstu systému a obtížnější oddělení jednotlivých částí aplikace.

Určitou variantu tohoto přístupu představuje modulární monolit. I v tomto případě jde o jednu aplikaci, její vnitřní struktura je však rozdělena do samostatných modulů s jasně vymezenými odpovědnostmi. Takové řešení umožňuje zachovat jednoduchost nasazení a současně zlepšit organizaci systému, jeho přehlednost a udržitelnost. Jednotlivé části aplikace mohou být navrženy tak, aby každá řešila konkrétní oblast funkcionality, aniž by docházelo k jejich zbytečnému provázání.

Další možností je mikroslužbová architektura, která rozděluje systém na více samostatně provozovaných služeb. Tento přístup může být výhodný u rozsáhlých systémů s vysokými nároky na škálování, samostatné nasazování jednotlivých částí nebo oddělenou správu více týmů. Současně však přináší vyšší technickou složitost, protože vyžaduje řešení komunikace mezi službami, správu distribuovaného prostředí a větší provozní nároky.

Pro aplikaci menšího až středního rozsahu, jejímž cílem je evidence pracovní doby a generování přehledných výstupů, se jako vhodnější jeví jednodušší architektonické řešení. V takovém případě je důležité především zachovat přehlednost, snadnou údržbu a nízkou provozní náročnost, nikoli budovat distribuovaný systém s vlastnostmi, které pro daný účel nejsou nezbytné.

1.4.5 Zdůvodnění zvoleného řešení

S ohledem na požadavky uvedené v kapitole 1.3 je pro navrženou aplikaci vhodné zvolit architekturu, která bude podporovat jednoduchost, přehlednost a oddělení jednotlivých částí systému. Z hlediska struktury aplikace je proto vhodným řešením modulární monolit, který umožňuje rozdělit systém do logických celků, aniž by bylo nutné zavádět složitou distribuovanou infrastrukturu. Takové řešení odpovídá charakteru navrhované aplikace a současně vytváří dostatečný prostor pro její další rozvoj.

V rámci prezentační vrstvy je vhodné využít návrhový vzor MVVM, protože tento přístup přirozeně podporuje oddělení uživatelského rozhraní od aplikační logiky. To je důležité zejména z hlediska přehlednosti kódu, snadnější údržby a možnosti budoucích úprav bez zásahu do všech částí systému. Kombinace modulárního monolitu a vzoru MVVM tak představuje řešení, které odpovídá požadavkům na funkčnost i kvalitu navržené aplikace.

Na základě této architektonické volby lze v další kapitole přistoupit k představení konkrétních technologií, které byly pro implementaci aplikace vybrány.

1.5 Použité technologie

Na základě zvolené architektury a návrhových principů je v dalším kroku vhodné vymezit technologie, které byly pro realizaci navrženého řešení použity. Volba konkrétních technologií má přímý vliv na způsob implementace aplikace, její přenositelnost, udržitelnost i možnosti dalšího rozvoje. V případě aplikace zaměřené na evidenci pracovní doby je důležité zvolit takové technologické prostředky, které umožní vytvořit přehledné uživatelské rozhraní, spolehlivě zpracovávat data a generovat odpovídající výstupy.

S ohledem na požadavky formulované v předchozích kapitolách byly pro realizaci aplikace zvoleny technologie platformy .NET, programovací jazyk C#, databázový systém SQLite a framework Avalonia UI. Tyto prostředky společně vytvářejí vhodný základ pro tvorbu desktopové aplikace, která je přehledná, snadno rozšiřitelná a současně technicky nenáročná na nasazení.

1.5.1 Platforma .NET a jazyk C#

Platforma .NET představuje moderní vývojové prostředí určené pro tvorbu různých typů aplikací, včetně desktopových, webových i mobilních řešení. Její výhodou je především široká podpora nástrojů, stabilní ekosystém a možnost vytvářet přenositelná řešení pro více operačních systémů. Z hlediska této práce je důležité zejména to, že .NET poskytuje vhodné prostředí pro vývoj aplikace, která kombinuje práci s daty, grafické uživatelské rozhraní a generování výstupních dokumentů.

Programovací jazyk C# byl zvolen především pro svou čitelnost, silnou typovou kontrolu a dobrou podporu objektově orientovaného přístupu. Tyto vlastnosti přispívají k přehlednější struktuře kódu, usnadňují jeho údržbu a snižují riziko některých typů chyb při vývoji. Výhodou C# je rovněž dobrá integrace s ostatními částmi platformy .NET, což umožňuje efektivně propojit jednotlivé vrstvy aplikace do jednoho funkčního celku.

Pro navržené řešení je tato technologie vhodná také proto, že podporuje tvorbu aplikací založených na architektonickém vzoru MVVM. Díky tomu je možné lépe oddělit prezentační vrstvu od aplikační logiky, což odpovídá požadavkům na přehlednost, udržitelnost a testovatelnost systému.

1.5.2 Databázová vrstva: SQLite a jazyk SQL

Součástí navrženého řešení je také práce s daty, která je potřeba ukládat, načítat a dále zpracovávat. Pro tento účel byla zvolena databáze SQLite, tedy lehký relační databázový systém určený zejména pro lokální ukládání dat v rámci jedné aplikace. Jeho výhodou je

jednoduché nasazení, protože nevyžaduje samostatný databázový server ani složitou správu databázového prostředí.

SQLite je vhodnou volbou zejména pro aplikace menšího a středního rozsahu, u nichž je důležitá spolehlivost, jednoduchost a nízké provozní nároky. V kontextu této práce představuje vhodné řešení pro ukládání údajů souvisejících s evidencí pracovní doby, protože umožňuje uchovávat strukturovaná data v přehledné podobě a současně poskytuje dostatečné možnosti pro jejich následné vyhledávání a zpracování.

Při práci s databází je využíván jazyk SQL, který slouží k definici datových struktur i k manipulaci s uloženými daty. SQL umožňuje vytvářet tabulky, upravovat obsah databáze a získávat data podle zadaných podmínek. Pro potřeby této práce je důležitý zejména proto, že poskytuje standardizovaný a srozumitelný způsob práce s relačními daty

1.5.3 Framework Avalonia UI

Pro tvorbu grafického uživatelského rozhraní byl zvolen framework Avalonia UI. Jedná se o moderní framework pro vývoj desktopových aplikací na platformě .NET, který umožňuje vytvářet multiplatformní uživatelská rozhraní s využitím deklarativního přístupu. To znamená, že vzhled aplikace je možné definovat odděleně od její logiky, což podporuje přehlednost a lepší organizaci celého projektu.

Významnou vlastností frameworku Avalonia UI je jeho přirozená podpora architektonického vzoru MVVM. Díky tomu je možné navrhnout uživatelské rozhraní tak, aby bylo odděleno od logiky zpracování dat a jednotlivé vrstvy aplikace nebyly zbytečně provázané. Tento přístup je pro navržené řešení vhodný zejména proto, že aplikace pracuje s větším množstvím údajů, formulářových prvků a výstupních přehledů, u nichž je důležitá přehledná správa stavu a jednoznačné rozdělení odpovědností.

Další výhodou Avalonia UI je možnost vytvářet jednotné uživatelské prostředí napříč více operačními systémy. To odpovídá požadavku na přenositelnost aplikace a současně umožňuje zachovat jednotný způsob ovládání a zobrazení dat bez ohledu na konkrétní platformu.

1.5.4 Podpůrné knihovny

Kromě hlavních technologií byly v projektu využity také podpůrné knihovny, které rozšiřují základní funkčnost aplikace a usnadňují implementaci vybraných úloh. Tyto knihovny nejsou samostatným jádrem řešení, ale představují doplňkové prostředky, které podporují práci s uživatelským rozhraním, zpracováním dat a tvorbou výstupů.

V souvislosti s architekturou MVVM mají význam zejména knihovny podporující práci se stavem aplikace, vazbami mezi daty a uživatelským rozhraním nebo zpracování uživatelských akcí. Dále jsou důležité knihovny určené pro práci se vstupními a výstupními formáty dat, například při načítání externích souborů nebo při vytváření výsledných dokumentů. Využití těchto prostředků umožňuje soustředit se na vlastní logiku aplikace a současně snižuje potřebu implementovat běžné technické mechanismy od začátku.

Při výběru podpůrných knihoven je důležité zohlednit jejich kompatibilitu se zvolenou architekturou, dlouhodobou udržitelnost a vhodnost pro daný typ aplikace. Jejich použití by mělo přispívat ke zvýšení kvality výsledného řešení, nikoli ke zbytečnému komplikování jeho struktury.

1.5.5 Shrnutí použitých technologií

Z výše uvedeného přehledu vyplývá, že zvolené technologie vytvářejí vhodný základ pro realizaci navrženého softwarového řešení. Platforma .NET spolu s programovacím jazykem C# umožňuje vývoj přehledné a udržitelné aplikace, SQLite poskytuje jednoduché a spolehlivé prostředí pro lokální správu dat a Avalonia UI podporuje tvorbu multiplatformního uživatelského rozhraní odpovídajícího zvolenému architektonickému přístupu. Podpůrné knihovny doplňují základní technologické prostředky o další funkce potřebné pro implementaci aplikace.

Na tuto kapitolu přirozeně navazuje představení vývojového prostředí a dalších nástrojů, které byly při implementaci využity.

1.6 Vývojové prostředí a podpůrné nástroje

Vedle volby vhodné architektury a technologií je pro realizaci softwarového řešení důležité také vývojové prostředí a podpůrné nástroje, které usnadňují implementaci, ladění a správu projektu. Tyto prostředky sice netvoří samotný základ navržené aplikace, ale mají významný vliv na efektivitu vývoje, přehlednost práce se zdrojovým kódem a možnost ověřování správnosti jednotlivých částí systému.

V rámci této práce byly využity nástroje, které podporují vývoj aplikace na platformě .NET, práci s databází SQLite a průběžnou kontrolu vytvářeného řešení. Jejich výběr vychází z požadavku na přehledné, stabilní a opakovatelné vývojové prostředí, které umožňuje efektivně realizovat jednotlivé části navržené aplikace.

1.6.1 Microsoft Visual Studio

Pro vývoj aplikace bylo využito integrované vývojové prostředí Microsoft Visual Studio. Jedná se o nástroj určený pro tvorbu aplikací na platformě .NET, který poskytuje prostředky pro psaní zdrojového kódu, správu projektové struktury, kompilaci, ladění i testování aplikace. Výhodou tohoto prostředí je zejména jeho komplexnost, díky níž lze většinu vývojových činností provádět v jednom sjednoceném rozhraní.

Microsoft Visual Studio podporuje práci s jazykem C# a poskytuje funkce, které usnadňují orientaci ve zdrojovém kódu, odhalování chyb a průběžné ověřování správnosti implementace. Důležitou vlastností je také podpora správy externích balíčků a knihoven, které jsou v projektu využívány. Tím je zajištěna lepší přehlednost závislostí a snadnější správa použitých komponent.

V kontextu této práce je Visual Studio vhodným nástrojem také proto, že podporuje vývoj desktopových aplikací na platformě .NET a umožňuje efektivní práci s více částmi projektu, například s logikou aplikace, datovou vrstvou i uživatelským rozhraním. Jeho využití tak přispívá k přehlednosti vývoje a k rychlejšímu ověřování navrženého řešení.

1.6.2 SQLiteStudio

Pro práci s databází byl využit nástroj SQLiteStudio, který slouží ke správě a kontrole databázových souborů systému SQLite. Tento nástroj umožňuje přehledné zobrazení databázové struktury, práci s tabulkami, spouštění SQL dotazů a ověřování uložených dat mimo samotnou aplikaci.

Význam SQLiteStudio spočívá především v tom, že usnadňuje kontrolu databázového schématu a obsahu databáze během vývoje. Díky tomu je možné jednoduše ověřovat správnost navržených tabulek, kontrolovat uložené záznamy a testovat databázové operace nezávisle na uživatelském rozhraní aplikace. Tento postup přispívá k lepší přehlednosti vývoje a usnadňuje identifikaci případných chyb při práci s daty.

V rámci této práce představuje SQLiteStudio vhodný podpůrný nástroj zejména proto, že doplňuje hlavní vývojové prostředí o možnost přímé práce s databází. Tím usnadňuje nejen návrh a kontrolu datové vrstvy, ale také průběžné ověřování správného fungování celé aplikace.

1.6.3 Shrnutí vývojového prostředí a podpůrných nástrojů

Zvolené vývojové prostředí a podpůrné nástroje vytvářejí vhodné podmínky pro realizaci navrženého softwarového řešení. Microsoft Visual Studio poskytuje komplexní prostředí pro implementaci, ladění a správu projektu, zatímco SQLiteStudio umožňuje efektivní kontrolu a

správu databázové vrstvy. Jejich kombinace přispívá k přehlednosti vývoje, usnadňuje ověřování jednotlivých částí systému a podporuje stabilní realizaci aplikace.

Na teoretickou část práce navazuje praktická část, která se již zaměřuje na samotný návrh, implementaci a ověření vytvořeného softwarového řešení.

2 PRAKTICKÁ ČÁST

Praktická část této práce se zaměřuje na návrh, implementaci a ověření aplikace určené pro evidenci pracovní doby. Návrh řešení vychází z požadavků formulovaných v teoretické části, zejména z požadavku na přehlednost, spolehlivost, snadnou použitelnost a omezení chybovosti při zpracování údajů. Cílem praktické části je ukázat, jak byly teoretické poznatky převedeny do konkrétního softwarového návrhu a jak byla vytvořena aplikace, která umožňuje práci se záznamy pracovní doby, jejich úpravu, zpracování vstupních dat a generování výsledných výstupů.

V následujících kapitolách je popsán cíl aplikace, její vnitřní struktura, organizace zdrojových souborů, návrh databázové vrstvy a podoba uživatelského rozhraní. Samostatná část je věnována algoritmům, které zajišťují zpracování rozvrhu a vyrovnávání pracovní doby. Závěr praktické části se zaměřuje na testování aplikace a ověření správnosti jejího fungování.

2.1 Cíl a návrh aplikace

Cílem navržené aplikace je usnadnit evidenci pracovní doby a zpřehlednit práci s jednotlivými záznamy tak, aby bylo možné omezit ruční zásahy, snížit riziko chyb a zjednodušit vytváření výsledných přehledů. Aplikace je určena pro práci s údaji souvisejícími s pracovním časem, jejich úpravou, kontrolou a následným převodem do výstupní podoby vhodné pro další administrativní využití.

Při návrhu aplikace byl kladen důraz na to, aby jednotlivé části systému byly logicky odděleny a aby bylo možné samostatně řešit práci s daty, aplikační logiku a uživatelské rozhraní. Takový přístup odpovídá požadavkům formulovaným v teoretické části práce a současně podporuje přehlednost, udržitelnost a možnost dalšího rozvoje aplikace.

Navržené řešení umožňuje načítání vstupních dat, práci se záznamy pracovní doby, jejich úpravu, validaci a následné vytváření přehledných výstupů. Důležitým aspektem návrhu bylo rovněž zajištění takového uživatelského prostředí, které bude srozumitelné a umožní efektivní práci bez zbytečně složitých kroků. Součástí návrhu je také zohlednění algoritmického zpracování údajů, které je nezbytné pro správné vyhodnocení a uspořádání evidence pracovní doby.

2.2 Struktura aplikace

Aplikace je navržena jako desktopové softwarové řešení, které pracuje s lokálně uloženými daty a poskytuje uživateli grafické rozhraní pro správu evidence pracovní doby. Z hlediska vnitřní organizace je rozdělena do několika vzájemně provázaných částí, z nichž každá plní specifickou

úlohu. Toto členění vychází ze zvoleného architektonického přístupu a umožňuje oddělit zpracování dat, aplikační logiku a prezentační vrstvu.

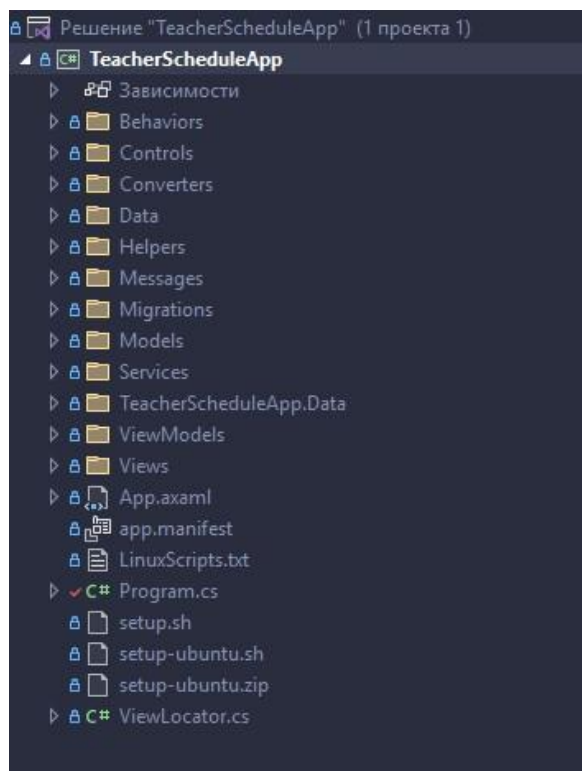
Základ systému tvoří datová vrstva, která zajišťuje ukládání a načítání údajů potřebných pro chod aplikace. Na ni navazuje aplikační logika, jejímž úkolem je zpracování vstupních dat, provádění kontrol, vyhodnocování záznamů a příprava údajů pro jejich zobrazení. Samostatnou část představuje vrstva uživatelského rozhraní, která zprostředkovává komunikaci mezi uživatelem a aplikací a umožňuje práci s jednotlivými funkcemi systému.

Důležitou součástí aplikace jsou také podpůrné služby, které zajišťují specifické operace, například import dat, validaci, zpracování vybraných pravidel evidence pracovní doby nebo generování výstupních dokumentů. Tím je dosaženo lepšího oddělení odpovědností mezi jednotlivými částmi systému a současně je usnadněna orientace ve zdrojovém kódu i případné budoucí rozšíření aplikace.

Takto navržená struktura odpovídá požadavku na přehlednost a udržitelnost řešení. Současně vytváří vhodný základ pro implementaci funkcí, které jsou pro evidenci pracovní doby klíčové, a umožňuje návazně popsat organizaci zdrojových souborů, databázovou strukturu a jednotlivé části uživatelského rozhraní.

2.3 Souborová struktura aplikace

Souborová struktura projektu byla navržena tak, aby odpovídala rozdělení aplikace do logických částí a podporovala přehlednost zdrojového kódu. Jednotlivé složky sdružují soubory podle jejich funkce v systému, například podle vztahu k uživatelskému rozhraní, datové vrstvě, aplikační logice nebo podpůrným mechanismům. Takové uspořádání usnadňuje orientaci v projektu, zjednodušuje údržbu a podporuje další rozšiřování aplikace.



Obrázek 4 Souborová struktura aplikace

2.3.1 Složka Behaviors

Složka Behaviors obsahuje prvky určené pro rozšíření chování uživatelského rozhraní bez nutnosti zasahovat přímo do logiky jednotlivých zobrazení. Tyto komponenty slouží zejména k zachycení nebo úpravě vybraných interakcí mezi uživatelem a aplikací.

2.3.2 Složka Controls

Složka Controls zahrnuje vlastní uživatelské ovládací prvky, které jsou opakovaně využívány v různých částech aplikace. Jejich účelem je sjednotit vzhled a chování vybraných částí rozhraní a omezit duplicitu v definici uživatelských prvků.

2.3.3 Složka Converters

Složka Converters obsahuje převodníky používané při vazbě dat mezi aplikační logikou a uživatelským rozhraním. Jejich úkolem je upravovat datové hodnoty do podoby vhodné pro zobrazení nebo další zpracování v rámci rozhraní aplikace.

2.3.4 Složka Data

Zde je datový kontext *AppDbContext*. Definuje přístup k databázi přes Entity Framework Core, entity pro události a nastavení a připojení k souboru SQLite v uživatelském profilu. Řeší mapování vlastností na sloupce a sjednocuje ukládání časových hodnot pomocí konvertorů.

```

namespace TeacherScheduleApp.Data
{
    Ссылка: 42 | Egor Razboinikov, 228 дн. назад | Автор: 1, изменений: 3
    public class AppDbContext : DbContext
    {
        Ссылка: 4 | 0 изменений | 0 авторов, 0 изменений
        public DbSet<Employee> Employees { get; set; }
        Ссылка: 6 | 0 изменений | 0 авторов, 0 изменений
        public DbSet<SemesterSettings> SemesterSettings { get; set; }
        Ссылка: 4 | 0 изменений | 0 авторов, 0 изменений
        public DbSet<WeekdaySettings> WeekdaySettings { get; set; }
        Ссылка: 7 | 0 изменений | 0 авторов, 0 изменений
        public DbSet<DaySettings> DaySettings { get; set; }
        Ссылка: 2 | 0 изменений | 0 авторов, 0 изменений
        public DbSet<ImportBatch> ImportBatches { get; set; }
        Ссылка: 41 | Egor Razboinikov, 337 дн. назад | Автор: 1, изменение: 1
        public DbSet<Event> Events { get; set; }

        Ссылка: 0 | Egor Razboinikov, 337 дн. назад | Автор: 1, изменение: 1
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            if (optionsBuilder.IsConfigured)
                return;

            string baseDir;
            if (RuntimeInformation.IsOSPlatform(OSPlatform.Windows))
                baseDir = Environment.GetFolderPath(Environment.SpecialFolder.ApplicationData);
            else
                baseDir = Path.Combine(
                    Environment.GetFolderPath(Environment.SpecialFolder.UserProfile),
                    ".config");

            var appDir = Path.Combine(baseDir, "TeacherScheduleApp");
            Directory.CreateDirectory(appDir);

            var dbPath = Path.Combine(appDir, "teacherapp.db");
            optionsBuilder.UseSqlite($"Data Source={dbPath}");
        }

        Ссылка: 0 | Egor Razboinikov, 228 дн. назад | Автор: 1, изменений: 3
        protected override void OnModelCreating(ModelBuilder modelBuilder) ...
    }
}

```

Obrázek 5 Ukázka kódu třídy AppDbContext

2.3.5 Složka Helpers

Složka Helpers obsahuje pomocné třídy a utility, které zajišťují dílčí podpůrné funkce využívané v různých částech aplikace. Tyto prvky nepředstavují samostatnou aplikační logiku, ale usnadňují implementaci opakujících se operací.

2.3.6 Složka Messages

Složka Messages slouží k definici zpráv a komunikačních objektů používaných při předávání informací mezi jednotlivými částmi aplikace. Jejím účelem je podpořit oddělení odpovědností a zjednodušit interní komunikaci systému.

2.3.7 Složka Models

Složka Models obsahuje datové modely reprezentující základní objekty aplikace. Tyto modely slouží k uchování a předávání údajů, se kterými aplikace pracuje v rámci evidence pracovní doby.

2.3.8 Složka Services

Složka Services zahrnuje služby zajišťující hlavní aplikační operace, například zpracování vstupních dat, validaci, import nebo generování výstupů. Tato vrstva soustřeďuje logiku, která není přímo vázána na uživatelské rozhraní.

2.3.9 Složka ViewModels

Složka ViewModels obsahuje prvky prezentační logiky, které zprostředkovávají komunikaci mezi datovou vrstvou a uživatelským rozhraním. ViewModely připravují data pro zobrazení, reagují na uživatelské akce a řídí chování jednotlivých obrazovek.

2.3.10 Složka Views

Složka Views zahrnuje jednotlivá zobrazení uživatelského rozhraní aplikace. Obsahuje definici oken, dialogů a dalších obrazovek, prostřednictvím kterých uživatel pracuje s funkcemi systému.

2.3.11 Složka Kořenové soubory projektu

Kořenové soubory projektu obsahují základní konfigurační a spouštěcí části aplikace. Patří sem zejména soubory potřebné pro sestavení projektu, inicializaci aplikace a správu jejích hlavních nastavení.

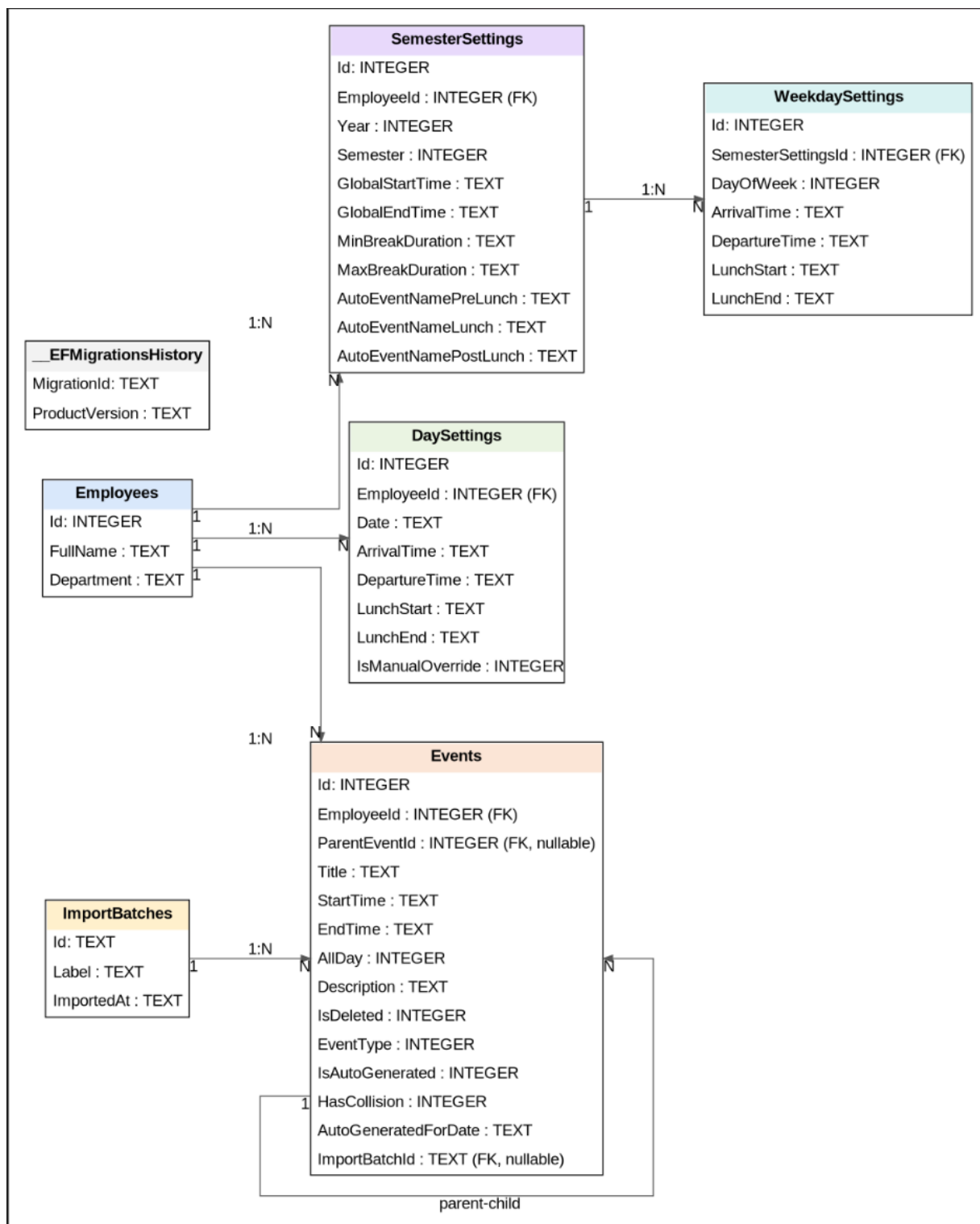
2.4 Struktura databáze

Databázová vrstva aplikace byla navržena jako relační model ukládaný v systému SQLite. Struktura databáze vychází z požadavků na evidenci pracovní doby, správu událostí a uchování nastavení souvisejících s pracovním režimem zaměstnance. Návrh databáze byl zaměřen na přehlednost, jednoznačné oddělení jednotlivých typů dat a podporu vazeb mezi hlavními entitami systému.

Centrální roli v databázovém modelu zaujímá tabulka **Employees**, která reprezentuje evidované zaměstnance. Na tuto tabulku jsou navázány další části databáze. Tabulka **Events** slouží k ukládání jednotlivých událostí souvisejících s evidencí pracovní doby a obsahuje jak vazbu na zaměstnance, tak i volitelnou vazbu na nadřazenou událost a importní dávku. Tabulka **DaySettings** uchovává individuální nastavení pracovního dne pro konkrétní datum a zaměstnance.

Další část databázového modelu tvoří tabulky **SemesterSettings** a **WeekdaySettings**, které slouží pro ukládání nastavení pracovního režimu v rámci semestru a jednotlivých dnů v týdnu. Tabulka **ImportBatches** eviduje informace o importovaných dávkách dat, na které mohou být jednotlivé události navázány.

Vazby mezi tabulkami jsou realizovány pomocí cizích klíčů, které zajišťují referenční integritu databáze. Návrh databázové struktury je znázorněn na obrázku 6.

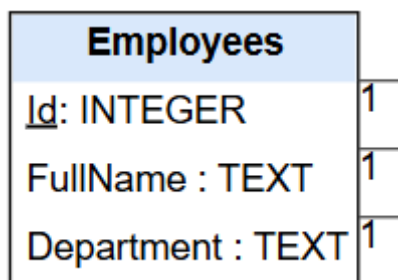


Obrázek 6 Relační schéma databáze

2.4.1 Přehled tabulek

Databázový model aplikace je tvořen několika vzájemně propojenými tabulkami, které společně zajišťují ukládání údajů o zaměstnancích, událostech, nastaveních pracovního režimu a importovaných datech. Jednotlivé tabulky byly navrženy tak, aby odpovídaly požadavkům aplikace na přehledné a spolehlivé zpracování evidence pracovní doby.

Tabulka **Employees** představuje základní entitu databázového modelu. Slouží k evidenci zaměstnanců, pro které jsou v systému vedeny záznamy pracovní doby a související nastavení. Obsahuje identifikační údaje zaměstnance, zejména jeho jméno a pracoviště. Na tuto tabulku jsou navázány další části databáze, protože každý záznam pracovní doby i nastavení pracovního režimu je vždy přiřazen konkrétnímu zaměstnanci.



Obrázek 7 Struktura tabulky Employees

Tabulka **Events** slouží k ukládání jednotlivých událostí souvisejících s evidencí pracovní doby. Jedná se o hlavní tabulku pro práci s časovými záznamy v aplikaci. Obsahuje název události, čas začátku a konce, informaci o celodenním charakteru události, popis, typ události a další příznaky související se stavem záznamu. Součástí tabulky jsou také údaje umožňující rozlišit, zda byla událost vytvořena automaticky, zda u ní byla zjištěna kolize, případně zda je navázána na nadřazenou událost nebo na konkrétní importní dávku. Díky této struktuře je možné v jedné tabulce uchovávat různé typy událostí souvisejících s pracovním režimem a současně zachovat jejich přehledné členění.

Events
<u>Id</u> : INTEGER
EmployeeId : INTEGER (FK)
ParentEventId : INTEGER (FK, nullable)
Title : TEXT
StartTime : TEXT
EndTime : TEXT
AllDay : INTEGER
Description : TEXT
IsDeleted : INTEGER
EventType : INTEGER
IsAutoGenerated : INTEGER
HasCollision : INTEGER
AutoGeneratedForDate : TEXT
ImportBatchId : TEXT (FK, nullable)

Obrázek 8 Struktura tabulky Events

Tabulka **DaySettings** slouží k ukládání individuálního nastavení pracovního dne pro konkrétní datum a zaměstnance. Obsahuje údaje o začátku a konci pracovního dne a dále časové údaje související s přestávkou na oběd. Důležitou součástí této tabulky je příznak **IsManualOverride**, který umožňuje rozlišit, zda bylo nastavení konkrétního dne upraveno ručně. Tato informace je významná zejména pro situace, kdy se denní nastavení odlišuje od výchozích pravidel stanovených pro delší časové období.

DaySettings
Id: INTEGER
EmployeeId : INTEGER (FK)
Date : TEXT
ArrivalTime : TEXT
DepartureTime : TEXT
LunchStart : TEXT
LunchEnd : TEXT
IsManualOverride : INTEGER

Obrázek 9 Struktura tabulky DaySettings

Tabulka **SemesterSettings** uchovává obecné nastavení pracovního režimu v rámci konkrétního semestru. Toto nastavení je navázáno na zaměstnance a obsahuje údaje, které určují výchozí parametry pracovního dne, například začátek a konec pracovní doby, minimální a maximální délku přestávky nebo názvy automaticky vytvářených bloků souvisejících s pracovním režimem. Tabulka tak představuje výchozí konfigurační základ, ze kterého může aplikace při zpracování dat vycházet.

SemesterSettings
Id: INTEGER
EmployeeId : INTEGER (FK)
Year : INTEGER
Semester : INTEGER
GlobalStartTime : TEXT
GlobalEndTime : TEXT
MinBreakDuration : TEXT
MaxBreakDuration : TEXT
AutoEventNamePreLunch : TEXT
AutoEventNameLunch : TEXT
AutoEventNamePostLunch : TEXT

Obrázek 10 Struktura tabulky SemesterSettings

Tabulka **WeekdaySettings** rozšiřuje semestrální nastavení o konkrétní konfiguraci jednotlivých dnů v týdnu. Je navázána na tabulku **SemesterSettings** a umožňuje definovat pracovní režim samostatně pro každý den. Obsahuje údaje o čase příchodu, odchodu a přestávky na oběd pro

zvolený den v týdnu. Díky tomu je možné zohlednit rozdíly mezi jednotlivými pracovními dny a přizpůsobit výchozí nastavení skutečnému rozvržení práce.

WeekdaySettings
Id: INTEGER
SemesterSettingsId : INTEGER (FK)
DayOfWeek : INTEGER
ArrivalTime : TEXT
DepartureTime : TEXT
LunchStart : TEXT
LunchEnd : TEXT

Obrázek 11 Struktura tabulky WeekdaySettings

Tabulka **ImportBatches** eviduje informace o dávkách importovaných dat. Slouží k uchování údajů o tom, kdy byl import proveden a pod jakým označením byl uložen. Tato tabulka umožňuje přiřadit jednotlivé importované události ke konkrétní dávce a tím usnadňuje dohledání původu dat i jejich následné zpracování.

ImportBatches
Id: TEXT
Label : TEXT
ImportedAt : TEXT

Obrázek 12 Struktura tabulky ImportBatches

Tabulka **__EFMigrationsHistory** je technická tabulka vytvářená frameworkem Entity Framework Core. Slouží k evidenci provedených migrací databázového schématu a umožňuje sledovat, jaké změny byly v databázi v průběhu vývoje aplikovány. Tato tabulka není součástí vlastní aplikační logiky, ale je důležitá z hlediska správy a údržby databázové struktury.

Z hlediska návrhu databáze lze říct, že jednotlivé tabulky pokrývají základní oblasti potřebné pro fungování aplikace. Datový model zahrnuje evidenci zaměstnanců, ukládání jednotlivých událostí, správu nastavení pracovního režimu i evidenci importovaných dat. Díky tomu vytváří vhodný základ pro další zpracování údajů v aplikační logice i pro jejich zobrazení v uživatelském rozhraní.

2.4.2 Shrnutí databázového návrhu

Navržená databázová struktura odpovídá požadavkům aplikace na ukládání údajů o zaměstnancích, událostech, nastavení pracovního režimu a importovaných datech. Jednotlivé

oblasti dat jsou rozděleny do samostatných tabulek, což přispívá k přehlednosti návrhu, jednodušší správě dat a lepší udržitelnosti celého řešení.

Důležitou vlastností databázového návrhu je také existence vazeb mezi hlavními entitami, které zajišťují referenční integritu a umožňují jednoznačné propojení jednotlivých částí systému. Navržená struktura tak vytváří vhodný základ pro implementaci aplikační logiky i pro následnou práci s uživatelským rozhraním aplikace.

2.5 Uživatelské rozhraní aplikace

Uživatelské rozhraní aplikace bylo navrženo s důrazem na přehlednost, srozumitelnost a jednoduchost ovládání. Při návrhu rozhraní bylo cílem vytvořit takové prostředí, ve kterém bude možné efektivně pracovat se záznamy pracovní doby, provádět jejich úpravy a současně zachovat dobrou orientaci v datech. Jednotlivé obrazovky aplikace proto vycházejí z jednotných principů ovládání a používají podobné vizuální i funkční prvky.

Rozhraní je členěno tak, aby uživatel mohl snadno přecházet mezi správou událostí, nastavením pracovního režimu a náhledem výsledných výstupů. Součástí návrhu bylo také zajištění návaznosti mezi jednotlivými pohledy, aby aplikace působila jako jeden ucelený systém, a ne jako soubor oddělených oken. Významnou roli hraje také vizuální prezentace dat, která umožňuje zobrazit evidenci pracovní doby v denním, týdenním a měsíčním pohledu.

2.5.1 Hlavní stránka aplikace

Hlavní stránka aplikace představuje výchozí pracovní prostředí, ve kterém uživatel provádí většinu běžných operací. Rozhraní hlavní stránky je rozděleno na dvě základní části. Levý panel slouží pro ovládání aplikace, výběr data, přístup k důležitým funkcím a zobrazení souhrnných informací. Pravá část okna je určena pro zobrazení kalendářového pohledu, ve kterém jsou prezentovány jednotlivé události a záznamy pracovní doby.

Na hlavní stránce jsou soustředěny funkce pro vytváření nových událostí, spuštění generování evidence pracovní doby, otevření globálního nastavení a zobrazení náhledu výstupního PDF dokumentu. Uživatel má současně možnost přepínat mezi denním, týdenním a měsíčním pohledem, čímž lze přizpůsobit zobrazení aktuální potřebě práce s daty. Součástí hlavní stránky je také kalendář pro rychlý výběr konkrétního dne a souhrnný přehled odpracovaného času vzhledem k požadované normě.

Návrh hlavní stránky vychází z požadavku na centralizaci nejdůležitějších funkcí do jednoho místa. Uživatel tak může provádět hlavní operace bez nutnosti složitého přecházení mezi různými částmi aplikace.

Vytvořit událost

Načíst rozvrh

Globální nastavení

EPD náhled

srpen 2025

^

∨

P

Ú

S

Č

P

S

N

28	29	30	31	1	2	3
4	5	6	7	8	9	10
11	12	13	14	15	16	17
18	19	20	21	22	23	24
25	26	27	28	29	30	31
1	2	3	4	5	6	7

Načtený soubory ∨

Rozvrh ∨

Nastavení pro den:

25.08.2025

Příchod:

08:30

Odchod:

17:00

Oběd od:

12:30

Oběd do:

13:00

Uložit nastavení

Obrázek 13 Levý panel aplikace

← Den: 25 srp 2025 → Dnes	
04:00	
05:00	
06:00	
07:00	
08:00	● Dopolodní pracovní doba
09:00	
10:00	
11:00	
12:00	
13:00	● Oběd ● Odpolední pracovní doba
14:00	
15:00	
16:00	
17:00	
18:00	

Obrázek 14 Pravý panel aplikace

2.5.2 Dialogové okno pro vytvoření události

Dialogové okno pro vytvoření události slouží k zadání nového záznamu do systému. Uživatel zde může vyplnit základní údaje o události, zejména její název, případný popis, časové vymezení a typ události. Rozhraní dialogu je navrženo tak, aby podporovalo jednoznačné a přehledné zadání všech potřebných údajů.

Součástí dialogu jsou prvky umožňující určit, zda se jedná o celodenní událost, a dále prvky pro zadání data a času jejího trvání. Uživatel také může vybrat typ události, což je důležité pro její další zpracování v rámci evidence pracovní doby. Při ukládání nového záznamu aplikace kontroluje vyplnění povinných údajů, čímž omezuje vznik neúplných nebo neplatných záznamů.

Toto dialogové okno představuje důležitý prvek rozhraní, protože umožňuje rozšiřovat evidenci pracovní doby o nové záznamy přímo z hlavního pracovního prostředí aplikace.

Nová událost

Název

Popis

☐ Celý den

Začátek:

4 květen 2025 0 00

Konec:

4 květen 2025 0 00

Typ události: Práce

Zrušit Vytvořit

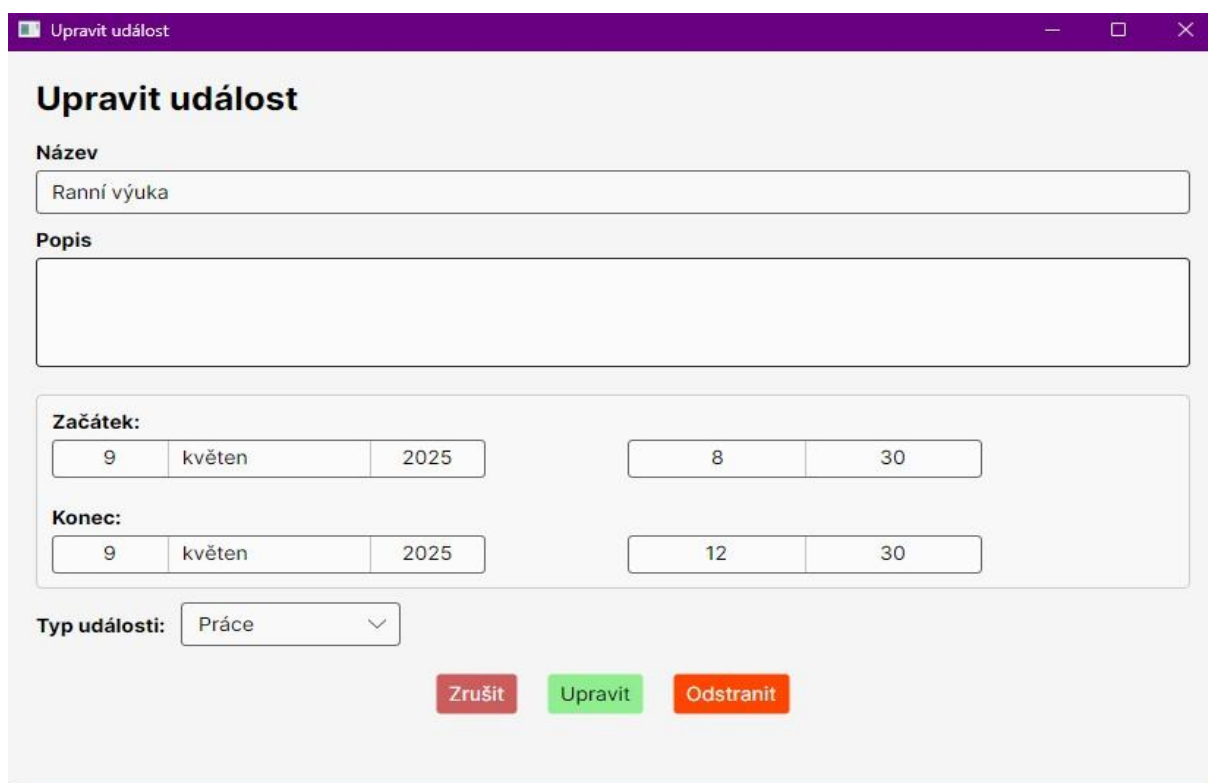
Obrázek 15 Dialogové okno pro vytvoření události

2.5.3 Dialogové okno pro změnu události

Dialogové okno pro změnu události vychází z obdobných principů jako dialog pro vytvoření nového záznamu, avšak je určeno pro úpravu již existujících údajů. Uživatel zde může měnit název události, její popis, časové vymezení i typ. Součástí dialogu je také možnost odstranění záznamu.

Návrh tohoto okna zajišťuje, aby bylo možné existující události upravovat přehledným a jednotným způsobem. Při změně údajů jsou opět kontrolovány povinné hodnoty a při odstraňování záznamu je vyžadováno potvrzení uživatele, aby se omezilo riziko nechtěného smazání.

Z hlediska uživatelského rozhraní je důležité, že vytváření a úprava událostí využívají podobný princip ovládání. Tím je zachována konzistence aplikace a uživatel nemusí při práci přecházet mezi odlišnými způsoby zadávání údajů.



Upravit událost

Název

Ranní výuka

Popis

Začátek:

9 květen 2025 8 30

Konec:

9 květen 2025 12 30

Typ události: Práce

Zrušit Upravit Odstranit

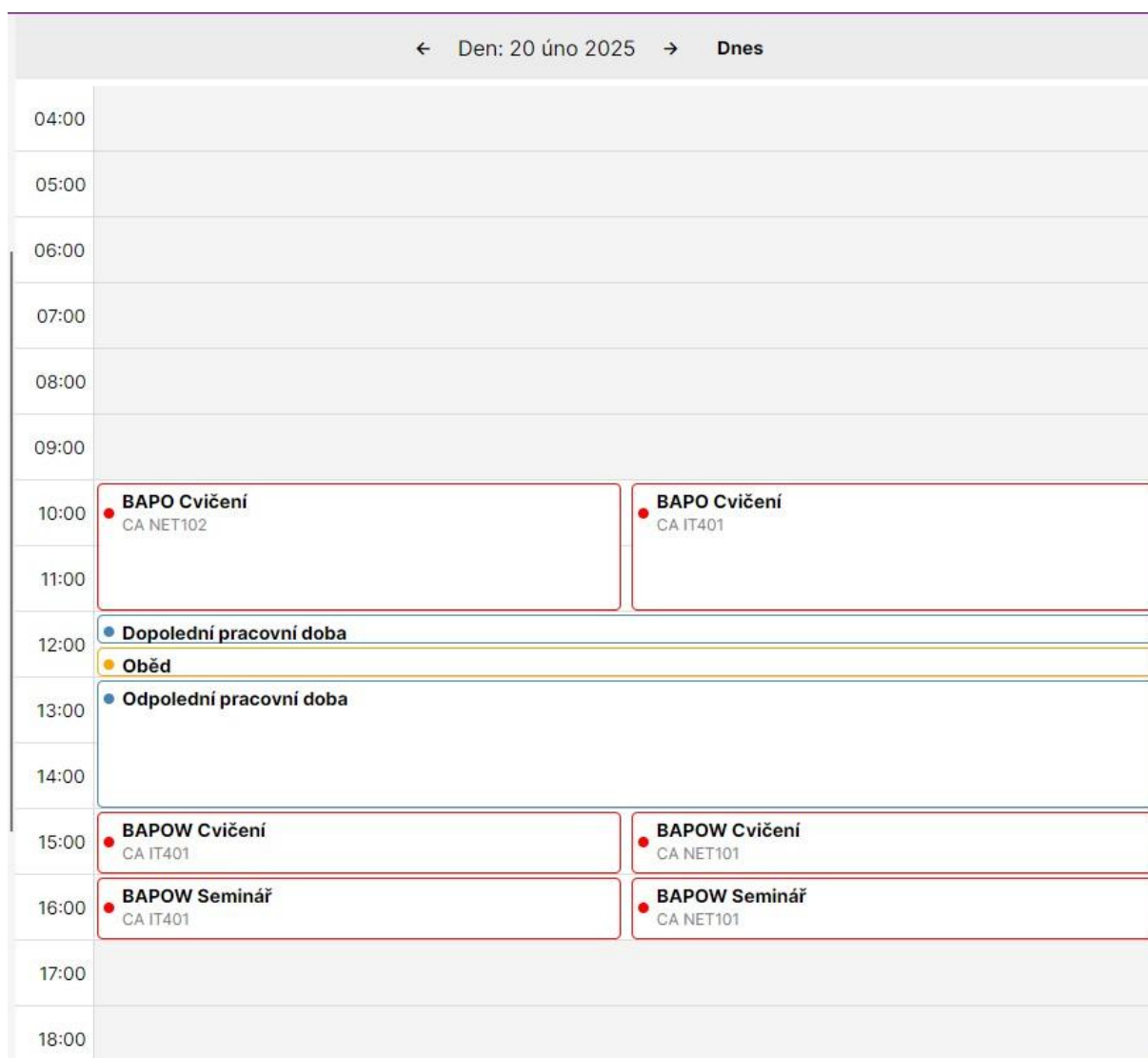
Obrázek 16 Dialogové okno pro změnu události

2.5.4 Vizuální prezentace pro den

Denní pohled slouží k detailnímu zobrazení konkrétního dne. Umožňuje zobrazit jednotlivé časové úseky v průběhu dne a v této struktuře prezentovat události, které se k danému datu vztahují. Toto zobrazení je vhodné zejména při práci s jednotlivými časově vymezenými záznamy nebo při kontrole konkrétního dne.

Součástí denního pohledu jsou také prvky pro přechod na předchozí a následující den a možnost návratu na aktuální datum. Vizuální členění zobrazení pomáhá odlišit pracovní intervaly od času mimo pracovní režim a umožňuje lepší orientaci v tom, jak jsou události v rámci dne rozloženy.

Denní pohled je významný zejména pro situace, kdy je třeba podrobně zkontrolovat nebo upravit konkrétní denní záznamy.

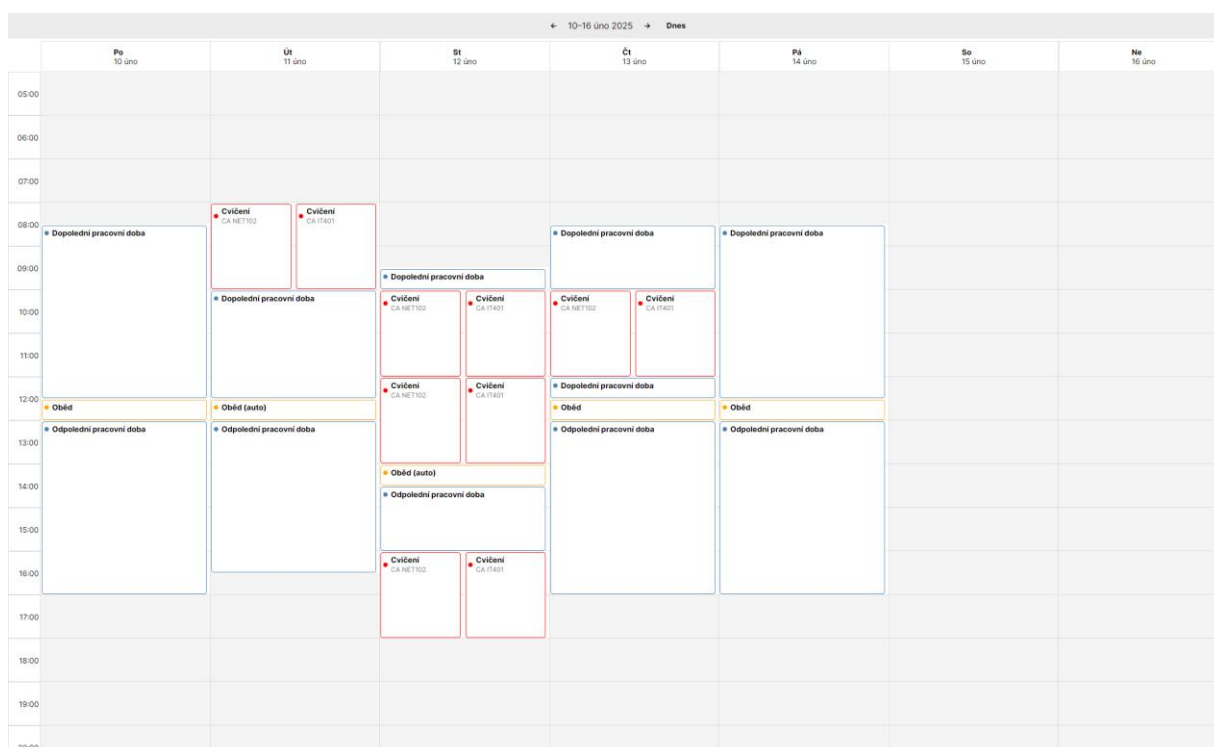


Obrázek 17 Vizualizace prezentace pro den

2.5.5 Vizualizace prezentace pro týden

Týdenní pohled rozšiřuje denní prezentaci na větší časový úsek a umožňuje sledovat události v rámci celého týdne. Toto zobrazení poskytuje přehled o rozložení pracovní doby v několika po sobě jdoucích dnech a usnadňuje orientaci v týdenním kontextu evidence.

Rozhraní týdenního pohledu obsahuje prvky pro přechod mezi jednotlivými týdny a možnost návratu na aktuální týden. Díky společnému zobrazení více dnů je možné snadněji porovnávat pracovní záznamy v rámci týdne a sledovat jejich vzájemné návaznosti. Tento pohled je proto vhodný zejména při posuzování rozložení pracovní doby v širším časovém rámci.



Obrázek 18 Vizuální prezentace pro týden

2.5.6 Vizuální prezentace pro měsíc

Měsíční pohled poskytuje nejširší kalendářový přehled a umožňuje zobrazit události v rámci celého měsíce. Zobrazení je založeno na mřížce kalendářních dnů, ve které jsou jednotlivé dny vizuálně odlišeny podle svého charakteru. Víkendy, státní svátky nebo dny náležející do předchozího či následujícího měsíce mohou být zvýrazněny odlišným způsobem, což podporuje rychlejší orientaci uživatele.

Stejně jako v ostatních pohledech je možné přecházet mezi časovými obdobími a vrátit se k aktuálnímu měsíci. Měsíční pohled je vhodný zejména pro celkový přehled o evidovaných událostech a pro orientační kontrolu rozložení pracovní doby v delším období.

Tento způsob prezentace představuje důležitou součást rozhraní, protože umožňuje uživateli sledovat evidenci pracovní doby nejen detailně, ale i v širším časovém kontextu.

← únor 2025 → Dnes						
Po	Út	St	Čt	Pá	So	Ne
28 08:00 Dopolední pracovní doba 12:00 Oběd	29 08:00 Dopolední pracovní doba 12:00 Oběd	30 08:00 Dopolední pracovní doba 12:00 Oběd	31 08:00 Dopolední pracovní doba 12:00 Oběd	01 08:00 Dopolední pracovní doba 12:00 Oběd	02	
03 08:00 Dopolední pracovní doba 12:00 Oběd	04 08:00 Dopolední pracovní doba 12:00 Oběd	05 08:00 Dopolední pracovní doba 12:00 Oběd	06 08:00 Dopolední pracovní doba 12:00 Oběd	07 08:00 Dopolední pracovní doba 12:00 Oběd	08	09
10 08:30 Dopolední pracovní doba 12:30 Oběd	11 08:00 Cvičení 08:00 Cvičení	12 09:30 Dopolední pracovní doba 10:00 Cvičení	13 08:30 Dopolední pracovní doba 10:00 Cvičení	14 08:30 Dopolední pracovní doba 12:30 Oběd	15	16
17 08:30 Dopolední pracovní doba 12:30 Oběd	18 08:00 Cvičení 08:00 Cvičení	19 09:30 Dopolední pracovní doba 10:00 Cvičení	20 08:30 Dopolední pracovní doba 10:00 Cvičení	21 08:30 Dopolední pracovní doba 12:30 Oběd	22	23
24 08:40 Dopolední pracovní doba 12:30 Oběd (auto)	25 08:00 Cvičení 08:00 Cvičení	26 08:30 oddáv 10:00 Cvičení	27 08:40 Dopolední pracovní doba 10:00 Cvičení	28 08:40 Dopolední pracovní doba 12:30 Oběd (auto)	29	30
03 08:30 Dopolední pracovní doba 12:30 Oběd	04 08:00 Cvičení 08:00 Cvičení	05 09:30 Dopolední pracovní doba 10:00 Cvičení	06 08:30 Dopolední pracovní doba 10:00 Cvičení	07 08:30 Dopolední pracovní doba 12:30 Oběd	08	09

Obrázek 19 Vizualní prezentace pro měsíc

2.5.7 Globální nastavení

Část globálního nastavení slouží ke konfiguraci údajů, které ovlivňují chování aplikace při zpracování evidence pracovní doby a při vytváření výsledných výstupů. Uživatel zde může upravovat výchozí časové parametry pracovního dne, nastavení přestávek a další údaje související s pracovním režimem.

Rozhraní je navrženo tak, aby bylo možné odděleně spravovat nastavení pro různá období, například pro jednotlivé semestry. Vedle časových parametrů obsahuje také údaje, které jsou využívány při tvorbě PDF výstupů nebo při automatickém vytváření vybraných typů událostí. Tato část aplikace je důležitá zejména proto, že umožňuje přizpůsobit chování systému konkrétním podmínkám bez zásahu do samotné implementace.

[← Zpět ke kalendáři](#)

Rok 2025 · Letní semestr

Rok: 2025

☐ Zimní

☒ Letní

Globální pracovní doba · Letní semestr

Od: 08:30

Do: 17:00

Pracovní doba · Letní semestr

Pondělí

Úterý

Středa

Čtvrtek

Pátek

Příchod: 08:30

Odchod: 17:00

Oběd, začátek: 12:30

Oběd, konec: 13:00

Další nastavení

Údaje pro PDF

Jméno: Radek Matoušek

Útvar: Katedra informačních technologií

Pauzy

Min. délka pauzy: 00:15

Max. délka pauzy: 01:00

Názvy událostí

Před obědem: Dopolední pracovní doba

Pro oběd: Oběd

Po obědě: Odpolední pracovní doba

Uložit nastavení

Obrázek 20 Globální nastavení

2.5.8 Ukázka PDF

Součástí uživatelského rozhraní je také obrazovka určená pro náhled výsledného PDF dokumentu. Tato část umožňuje uživateli zkontrolovat podobu výstupu ještě před jeho uložením. Náhled výstupu podporuje lepší kontrolu správnosti generovaného dokumentu a umožňuje ověřit, zda výsledná podoba odpovídá očekávání.

46

Uživatel zde může přepínat mezi jednotlivými měsíci a zvolit uložení dokumentu do počítače. Zařazení této funkce přímo do uživatelského rozhraní zvyšuje praktičnost aplikace, protože propojuje práci s evidencí pracovní doby a kontrolu finálního výstupu do jednoho prostředí.

07-2025

Uložit PDF

Evidence pracovní doby, včetně přestávek v práci a práce přesčas

Fakulta elektrotechniky a informatiky
Jméno: Radek Matoušek
útvár: Katedra informačních technologií

pracovní doba: 08:30–17:00 hod.
docházka za měsíc: červenec 2025

Fond prac. doby:
184 hodin

Den	Začátek pracovní doby	1. přestávka		2. přestávka		Konec pracovní doby	Odpracováno	Přesčas	Neodprac.	Poznámka)
		Od	Do	Od	Do					
1.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
2.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
3.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
4.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
5.										S
6.										S
7.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
8.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
9.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
10.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
11.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
12.										
13.										
14.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
15.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
16.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
17.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
18.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
19.										
20.										
21.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
22.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
23.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
24.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
25.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
26.										
27.										
28.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
29.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
30.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
31.	08:30	12:30	13:00			17:00	08:00:00	00:00:00	00:00:00	
Celkem							184:00:00	00:00:00	00:00:00	

kontr. č. Σ odprac. - přesčas + neodprac. hodin (odpovídá FPD): 184:00:00

podpis zaměstnance:

podpis nadřízeného pracovníka:

) do poznámky: D - dovolená, N - nemoc, OČR - ošetřování, L - lékař, PC - pracovní cesta, S - svátek a ostatní dny pracovního klidu

Obrázek 21 Ukázka PDF

2.5.9 Shrnutí kapitoly

Uživatelské rozhraní aplikace bylo navrženo tak, aby podporovalo přehlednou a efektivní práci s evidencí pracovní doby. Jednotlivé části rozhraní pokrývají hlavní činnosti uživatele, od správy událostí přes nastavení pracovního režimu až po kontrolu výsledných výstupů. Důraz byl kladen na jednotnost ovládání, návaznost jednotlivých obrazovek a srozumitelnou vizuální prezentaci dat.

Navržené rozhraní tak vytváří vhodné prostředí pro každodenní práci s aplikací a současně podporuje funkce, které jsou pro evidenci pracovní doby zásadní. Na tuto kapitolu navazuje část věnovaná algoritmům, které zajišťují zpracování vstupních dat a vyhodnocení evidence pracovní doby

2.6 Algoritmy

Tato kapitola se zaměřuje na algoritmy, které zajišťují klíčové zpracování dat v aplikaci. Zatímco předchozí kapitoly byly věnovány návrhu databázové struktury a uživatelského rozhraní, v této části je pozornost soustředěna na vnitřní logiku aplikace, tedy na způsob, jakým

jsou vstupní údaje vyhodnocovány, upravovány a převáděny do podoby použitelné pro evidenci pracovní doby.

Z hlediska funkčnosti aplikace mají zásadní význam dva procesy. Prvním z nich je algoritmus vyrovnávání hodin, jehož úkolem je upravit pracovní záznamy tak, aby odpovídaly požadované denní, týdenní a měsíční evidenci při zachování zadaných pravidel. Druhým procesem je algoritmus importu rozvrhu ze systému STAG, který převádí externí data ve formátu CSV do interní struktury událostí používané aplikací. Oba algoritmy jsou implementovány v aplikační logice systému a tvoří základ pro správné fungování celé aplikace. Podle zdrojového kódu je vyrovnávání řízeno zejména metodami *BalanceEventsForMonthAsync*, *BalanceOneWeekAsync*, *BalanceTwoWeeksAsync* a následnou normalizací měsíce, zatímco import využívá zpracování CSV, mazání předchozích importovaných záznamů a hromadné vytváření nových událostí.

Pro lepší srozumitelnost jsou oba algoritmy v následujících podkapitolách popsány nejprve z obecného hlediska a poté prostřednictvím hlavních kroků jejich zpracování. Součástí této kapitoly mohou být také blokové diagramy, které znázorňují jejich logickou strukturu a usnadňují orientaci v jednotlivých fázích zpracování.

2.6.1 Algoritmus vyrovnávání hodin

Algoritmus vyrovnávání hodin slouží k tomu, aby evidence pracovní doby odpovídala požadovanému pracovnímu fondu a současně respektovala pravidla, podle nichž mají být zachovány ručně zadané události. Jeho hlavním cílem je dosáhnout vyrovnání pracovní doby v rámci dne, týdne a měsíce, aniž by docházelo k neodůvodněným zásahům do uživatelem zadaných záznamů.

Základním principem algoritmu je rozlišení mezi ručními a automaticky generovanými událostmi. Ručně zadané události představují pevné vstupní údaje, které zůstávají zachovány, zatímco automaticky vytvářené pracovní bloky lze upravovat tak, aby bylo dosaženo požadovaného výsledku. Tím je zajištěno, že aplikace zasahuje pouze do těch částí evidence, které jsou určeny k automatickému zpracování. Toto pravidlo je patrné i ve zdrojovém kódu, kde se ořezávání a prodlužování provádí pouze na automaticky generovaných okrajích dne a ruční události zůstávají nedotčené.

Vyrovnávání probíhá na úrovni kalendářního měsíce. Nejprve jsou určeny pracovní dny příslušného období, přičemž se zohledňují víkendy a státní svátky. Následně jsou tyto dny rozděleny do jednotlivých týdnů a každý týden je zpracován samostatně. Pokud se na začátku

a na konci měsíce nacházejí neúplné týdny, algoritmus je může posuzovat společně, aby bylo dosaženo správného měsíčního vyrovnání. Po zpracování všech týdnů následuje dodatečná normalizace měsíce, která slouží k odstranění drobných odchylek. Tento postup odpovídá metodám *BalanceEventsForMonthAsync*, *BalanceOneWeekAsync*, *BalanceTwoWeeksAsync* a *PostNormalizeMonthAsync*.

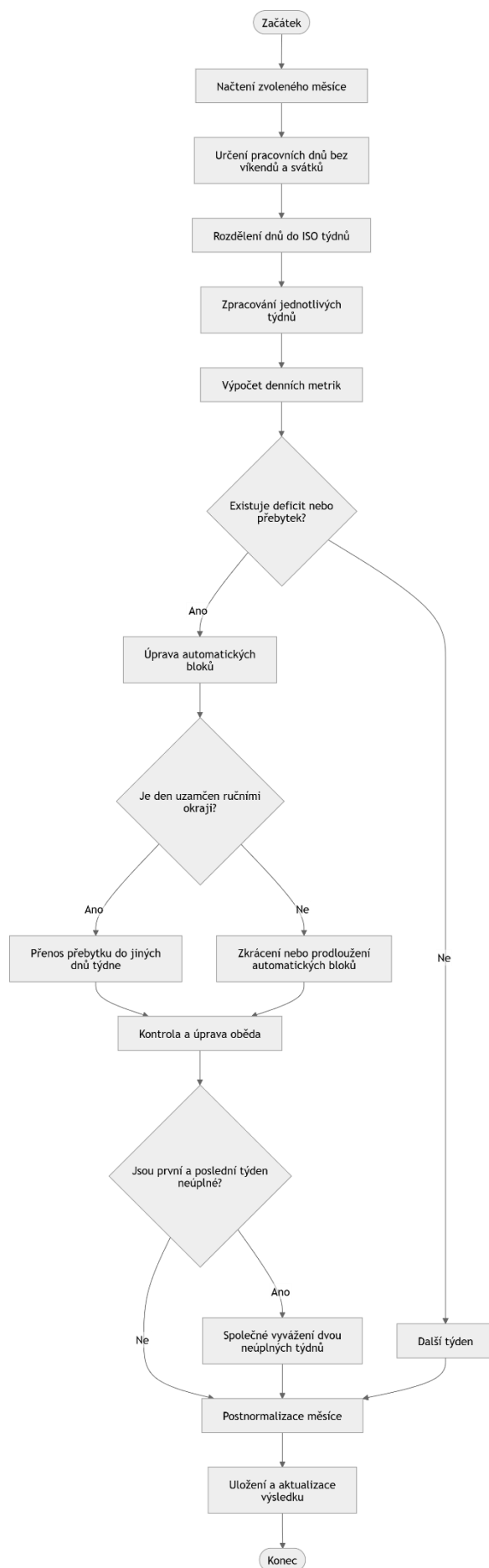
Pro jednotlivé dny jsou vypočítávány pracovní metriky, které rozlišují skutečně započítávanou práci a zvláštní typy nepřítomnosti nebo jiných uznávaných stavů. Na základě těchto metrik se pro každý den určuje, zda vykazuje deficit, nebo naopak přebytek odpracovaného času. Pokud je v některém dni vykázán přesah, algoritmus se jej nejprve pokouší odstranit v rámci téhož dne zkrácením automatických bloků na jeho začátku nebo na konci. Pokud jsou okraje dne tvořeny ručními událostmi a den je tak z hlediska automatických úprav „uzamčen“, může být přebytek přenesen do jiných dnů stejného týdne, které takovou úpravu umožňují. Tato logika je ve zdrojovém kódu zajišťována mimo jiné metodami *IsLockedEdgeDay*, *TransferLockedOvertimeAsync*, *TrimStartAuto* a *TrimEndAuto*.

Dalším úkolem algoritmu je párování dnů s přebytkem a dnů s deficitem. Pokud to situace dovoluje, jsou automatické pracovní bloky v některých dnech zkracovány a v jiných naopak prodlužovány tak, aby bylo dosaženo týdenního vyrovnání. Tyto úpravy jsou prováděny s ohledem na zachování konzistence denního rozvrhu a na správné umístění přestávky na oběd.

Samostatnou část algoritmu tvoří práce s obědovou přestávkou. Po každé významnější změně časových hranic dne je nutné ověřit, zda přestávka leží uvnitř pracovního okna a zda nekoliduje s ostatními událostmi. Pokud tomu tak není, algoritmus se pokouší nalézt nové vhodné umístění přestávky, případně ji vymezit uvnitř automaticky generovaných pracovních bloků. Ve zdrojovém kódu tuto část zajišťují zejména metody *EnsureLunchInsideWorkWindowAsync*, *RefitLunch*, *GetBusyIntervals*, *TryCarveLunchFromInnerAuto*, *ReplaceLunchEvent* a *SplitAutoWorkAroundLunch*.

Výsledkem algoritmu je taková podoba evidence pracovní doby, která respektuje ručně zadané události, upravuje pouze automaticky generované části dne a současně zachovává správné časové uspořádání včetně přestávky na oběd. Výhodou tohoto přístupu je předvídatelnost výsledku a možnost opakovaného spuštění nad stejnými daty bez vzniku nekonzistentních změn.

Pro větší přehlednost je logika algoritmu znázorněna na blokovém diagramu uvedeném na *obrázku 22*



2.6.2 Algoritmus importu rozvrhu ze STAG

Druhým klíčovým algoritmem aplikace je import rozvrhu ze systému STAG. Jeho úkolem je převést vstupní data uložená ve formátu CSV do podoby interních událostí, se kterými aplikace dále pracuje při evidenci pracovní doby. Importní algoritmus tak představuje vstupní bod pro automatizované načtení rozvrhových údajů a jejich následné začlenění do databázové struktury aplikace.

Import začíná načtením CSV souboru a kontrolou jeho základní struktury. Při zpracování jednotlivých řádků jsou postupně ověřovány hodnoty, jako jsou datumové intervaly, základní datum, čísla týdnů a časové údaje. Pokud některý řádek neobsahuje platné údaje, je přeskočen a zaznamenán jako chybový. Ve zdrojovém kódu lze tuto část sledovat v logice validace hlavičky a následného zpracování jednotlivých řádků, včetně metod *TryParseDate*, *TryParseTime* a *TryParseWeek*.

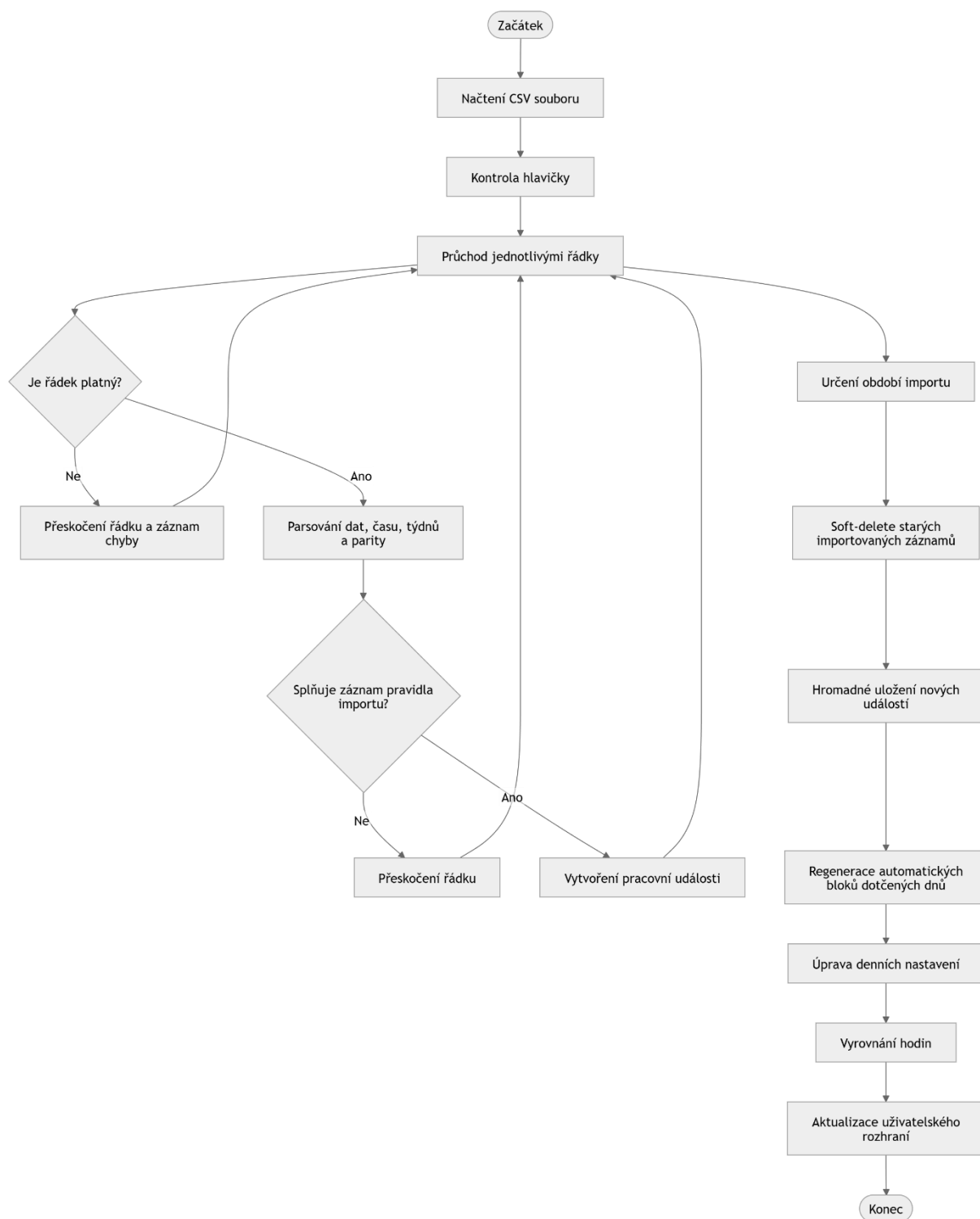
Před samotným uložením nových dat algoritmus určí časový rozsah importu a v tomto období označí dříve importované záznamy jako neaktivní. Tím je zajištěno, že nová dávka může bezpečně nahradit předchozí import bez nutnosti fyzického mazání starších údajů. Tato logika odpovídá metodě *BulkSoftDeleteImportedInRangeAsync*, která pracuje s importovanými událostmi navázanými na identifikátor dávky.

Následně algoritmus vytváří nové pracovní události na základě pravidel odvozených z CSV souboru. U každého řádku se zohledňuje den v týdnu, období platnosti, čas začátku a konce, rozsah týdnů a případně také parita týdnů. Událost je vytvořena pouze tehdy, pokud konkrétní datum splňuje všechny požadované podmínky. Takto vzniklé záznamy jsou ukládány jako ručně zadané pracovní události typu *Work* a jsou přiřazeny k aktuální importní dávce. Po zpracování všech řádků jsou nové události hromadně uloženy do databáze pomocí operace *CreateEventsBulk*.

Po uložení importovaných událostí následuje navazující fáze, jejímž účelem je převést importovaná data do podoby vhodné pro další práci v aplikaci. Pro všechny dotčené dny dochází k regeneraci automatických bloků, kontrole denního nastavení a následnému vyrovnání pracovní doby. Aplikace tak po importu nezůstává pouze u převodu dat, ale bezprostředně připravuje denní záznamy pro další zpracování, včetně kontroly přestávky a hran dne. Ve zdrojovém kódu je tato návaznost vidět na opakovaném volání *RegenerateDailyEventsAsync*, úpravách denních nastavení a následném vyrovnání změněného rozsahu.

Výsledkem importního algoritmu je sada nově založených pracovních událostí, které odpovídají datům načteným ze systému STAG a jsou zároveň připraveny pro další zpracování v rámci evidence pracovní doby. Výhodou tohoto postupu je možnost opakovaného importu, zachování původu dat prostřednictvím importních dávek a návaznost na další algoritmy aplikace.

Také tento algoritmus je pro větší názornost doplněn blokovým diagramem uvedeným na *obrázku 23*.



Obrázek 23 Algoritmus vyrovnávání hodin

2.7 Testování aplikace

Tato kapitola se zaměřuje na ověření funkčnosti vytvořené aplikace. Cílem testování bylo zjistit, zda aplikace správně podporuje celý pracovní postup, tedy import rozvrhu z externího souboru, následné zpracování a vyrovnání pracovní doby podle pravidel evidence pracovní doby, a nakonec vytvoření výsledného výstupu ve formátu PDF. Současně bylo ověřováno, zda aplikace

vykazuje stabilní chování na podporovaných platformách a zda je její uživatelské rozhraní použitelné i při delších nebo výpočetně náročnějších operacích.

Testování probíhalo průběžně v průběhu implementace i po dokončení hlavních funkcí aplikace. V závěrečné fázi byly provedeny souvislé testy nad vybranými scénáři, které pokrývaly běžné i hraniční situace. Pozornost byla věnována především správnosti denních, týdenních a měsíčních součtů, korektnímu chování algoritmu vyrovnávání, správnému zpracování importovaných dat a čitelnosti výsledných výstupů.

2.7.1 Cíl a kritéria přijetí

Hlavním cílem testování bylo ověřit, že aplikace odpovídá požadavkům stanoveným v předchozích kapitolách a že ji lze použít pro praktickou práci s evidencí pracovní doby. Za přijatelný výsledek bylo považováno takové chování systému, při němž aplikace bez chyby načte vstupní CSV soubor, vytvoří odpovídající události, správně provede jejich další zpracování a následně umožní vytvoření přehledného výstupu ve formátu PDF.

Dále bylo požadováno, aby algoritmus správně vyrovnával pracovní dobu v rámci týdne a současně korektně zohledňoval neúplné týdny na začátku a na konci měsíce. Výstupní PDF dokument musel obsahovat správně strukturovanou tabulku evidence pracovní doby, včetně součtů, hlaviček a zvýraznění relevantních dnů, například státních svátků. Současně bylo ověřováno, že aplikace vykazuje srovnatelné chování na operačních systémech Windows a Linux a že uživatelské rozhraní zůstává dostatečně responzivní i při delším zpracování.

2.7.2 Testovací prostředí

Aplikace byla testována na dvou hlavních platformách, a to na operačním systému Windows 11 a na systému Ubuntu 22.04 LTS. V obou případech byla použita platforma .NET 8, uživatelské rozhraní postavené na frameworku Avalonia UI a lokální databázové úložiště SQLite.

Na platformě Windows probíhalo testování v prostředí Windows 11 verze 23H2. Na platformě Linux bylo testování realizováno v prostředí Ubuntu 22.04 LTS, a to s běžnou systémovou konfigurací desktopového prostředí. V obou případech byla aplikace provozována nad reálnou databází SQLite uloženou v souboru a byly používány testovací vstupy odpovídající běžnému provozu aplikace.

2.7.3 Metodika testování

Pro ověření funkčnosti aplikace byla použita kombinace integračního a manuálního testování. Integrační testování bylo zaměřeno na celé zpracovatelské řetězce, tedy zejména na návaznost mezi importem dat, vytvořením a úpravou událostí, automatickým vyrovnáváním pracovní

doby a generováním PDF výstupů. Manuální testování bylo využito zejména při ověřování uživatelského rozhraní, správného zobrazení údajů a reakce aplikace na uživatelské vstupy.

Při testování byly používány jak validní vstupní soubory, tak záměrně upravené chybové vstupy. Tím bylo možné ověřit nejen běžné scénáře, ale také reakci aplikace na neplatné datумы, nesprávné formáty času, chybějící sloupce nebo kolizní situace v datech. U výsledných PDF dokumentů nebyla posuzována binární shoda souborů, ale obsahová správnost, přehlednost a úplnost výstupu.

Testování bylo zaměřeno jak na správnost datového zpracování, tak na použitelnost výsledného řešení v praxi. Ověřovány byly tedy nejen správné součty a výstupy, ale také plynulost práce s aplikací, srozumitelnost chybových hlášení a stabilita systému při opakovaném provádění stejných operací.

2.7.4 Předpokládané rozdíly mezi platformami

Při testování bylo zohledněno, že mezi platformami Windows a Linux mohou vznikat drobné vizuální rozdíly. Tyto rozdíly se mohou týkat zejména použitých systémových fontů, metrik textu, šířky některých prvků nebo drobných odlišností ve vykreslování uživatelského rozhraní. Podobně se mohou projevit i malé rozdíly ve vzhledu rámečků, posuvníků nebo dialogových prvků.

Za přijatelný stav bylo považováno, pokud tyto rozdíly neovlivní obsahovou správnost aplikace, čitelnost rozhraní ani použitelnost výstupních dokumentů. V praxi to znamená, že text musí být na obou platformách čitelný, ovládací prvky musí být dostupné a nesmí docházet k překrývání obsahu. U exportovaných PDF dokumentů musel zůstat zachován stejný obsah, i když se mohly mírně lišit některé vizuální detaily dané platformou.

2.7.5 Testovací scénáře

Testovací scénáře byly navrženy tak, aby pokrývaly běžné situace při používání aplikace i vybrané hraniční případy. Každý scénář vycházel ze stejného principu: nejprve byla připravena vstupní data nebo odpovídající stav databáze, následně byla provedena konkrétní akce v aplikaci a poté byl porovnán očekávaný a skutečný výsledek.

První skupinu scénářů tvořily testy zaměřené na standardní provoz aplikace. V těchto scénářích byl použit platný export CSV ze systému STAG za jeden kalendářní měsíc. Po dokončení importu bylo ověřováno, zda došlo ke správnému vytvoření událostí, zda byly správně vypočteny denní a týdenní součty a zda aplikace automaticky doplnila přestávku na oběd a upravila případné přesahy pouze v automaticky generovaných blocích. Na tento scénář

navazovalo ověření opakovaného importu stejného období, při němž bylo sledováno, zda nedochází ke vzniku duplicit a zda v databázi zůstává pouze aktuální importní dávka.

Další skupinu tvořily negativní scénáře. Zde byly záměrně připraveny neplatné nebo neúplné vstupy, například soubory s chybně zadaným datem, časem nebo chybějícím povinným sloupcem. V těchto případech bylo sledováno, zda aplikace správně reaguje chybovým hlášením nebo bezpečně přeskočí problematické řádky, aniž by došlo k porušení konzistence uložených dat.

Samostatná skupina testů byla zaměřena na situace související s pravidly evidence pracovní doby. Byly ověřovány scénáře zahrnující neúplné týdny, státní svátky, přesahy mezi začátkem a koncem měsíce, kombinaci výuky a služební cesty v jednom dni nebo krátkodobou nepřítomnost zasahující do pracovní doby. V těchto scénářích bylo důležité zejména potvrdit správnost výpočtu odpracovaného a neodpracovaného času a korektní chování algoritmu vyrovnávání.

Dále byly testovány i scénáře zaměřené na ruční zásahy uživatele, například změna okrajů dne nebo délky oběda. V těchto případech bylo sledováno, zda aplikace po úpravě správně regeneruje automatické bloky, zachová platné denní nastavení a nevytváří nekonzistentní záznamy.

Poslední skupina testů byla věnována exportu, výkonu a použitelnosti. Byla ověřována správnost PDF výstupu, rychlost importu a exportu, chování aplikace při zvětšeném měřítku uživatelského rozhraní, reakce na pokus o ukládání do chráněné složky a také stabilita aplikace při opakovaném importu stejného souboru.

2.7.6 Souhrnná tabulka testů

ID	Platforma	Testovaný scénář	Očekávaný výsledek	Výsledek
1	Windows / Linux	Import platného CSV ze STAGu	Události jsou vytvořeny, součty odpovídají očekávání, oběd je doplněn a přesahy jsou upraveny pouze	PASS

			v automatických blocích	
2	Windows / Linux	Opakovaný import stejného období	Nevzniknou duplicity a v databázi zůstane pouze aktuální importní dávka	PASS
3	Windows / Linux	CSV s chybně formátovaným datem nebo časem	Aplikace zobrazí řízenou chybu a nedojde k nekonzistentní změně dat	PASS
4	Windows / Linux	CSV s chybějícím povinným sloupcem	Problematické řádky jsou přeskočeny nebo je import korektně odmítnut, data zůstávají konzistentní	PASS
5	Windows / Linux	Překryv dvou bloků výuky v jednom dni	Překryv se nezapočítá dvakrát a kolize je jednoznačně označena	PASS
6	Windows / Linux	Neúplné týdny a státní svátky	Týdenní fond odpovídá počtu pracovních dnů a svátky snižují požadovaný fond	PASS

7	Windows / Linux	Vyrovnnání mezi prvním a posledním neúplným týdnem	Hodiny jsou správně dovršené bez zásahu do plných týdnů	PASS
8	Windows / Linux	Služební cesta a výuka v jednom dni	Služební cesta je správně započtena jako práce a překryv se nezdvouje	PASS
9	Windows / Linux	Krátká nepřítomnost přes výuku	Nepřítomnost je správně evidována a součty zůstávají korektní	PASS
10	Windows / Linux	Ruční editace a regenerace automatických bloků	Po zásahu uživatele dojde ke korektní regeneraci a zachování platných hran dne	PASS
11	Windows / Linux	Export PDF za měsíc	PDF obsahuje správné hlavičky, součty a označení relevantních dnů	PASS
12	Windows / Linux	Rychlost importu	Import proběhne v přijatelném čase a bez zablokování	PASS

			uživatelského rozhraní	
13	Windows / Linux	Rychlost generování PDF	PDF je vytvořeno v přijatelném čase a lze jej bez problémů otevřít	PASS
14	Windows / Linux	Škálování UI 125–150 %	Prvky rozhraní zůstávají čitelné a nepřekrývají se	PASS
15	Windows / Linux	Uložení do chráněné složky	Aplikace zobrazí srozumitelné chybové hlášení a neukončí se nekorektně	PASS
16	Windows / Linux	Data mimo zvolený měsíc	PDF obsahuje pouze dny příslušného měsíce a správné součty	PASS
17	Windows / Linux	Přerušení dlouhé operace	Operace je bezpečně ukončena a databáze zůstává v konzistentním stavu	PASS
18	Windows / Linux	Determinismus opakovaného importu	Opakovaný import téhož souboru vede ke	PASS

			stejnému výsledku	
--	--	--	----------------------	--

2.7.7 Shrnutí testování a známá omezení

Provedené testování ukázalo, že aplikace je schopna spolehlivě zvládnout hlavní pracovní scénáře, pro které byla navržena. Import rozvrhu, vyrovnávání pracovní doby i export PDF výstupů vykazovaly na platformách Windows i Linux srovnatelné chování. Zjištěné rozdíly mezi platformami měly převážně vizuální charakter a neovlivňovaly správnost dat ani použitelnost aplikace.

Výsledné PDF dokumenty se mohly v některých případech lišit v technických detailech, například v metadatech nebo drobných typografických vlastnostech, avšak jejich obsahová správnost zůstávala zachována. Z tohoto důvodu bylo při testování PDF dokumentů posuzováno především jejich věcné a vizuální vyznění, nikoli binární shoda souborů.

Za známá omezení lze považovat především závislost některých vizuálních detailů na konkrétní platformě a použitých systémových fontech. Tato omezení však nemají zásadní dopad na hlavní funkce aplikace. Pro budoucí rozvoj by bylo možné testovací část rozšířit o automatizované testy většího rozsahu, případně doplnit archiv testovacích vstupních souborů a obrazové záznamy vybraných kontrolních scénářů.

Z výsledků testování lze uzavřít, že aplikace splňuje stanovené požadavky a je použitelná pro praktickou práci s evidencí pracovní doby. Testy současně potvrdily správnost hlavních algoritmů, stabilitu uživatelského rozhraní a konzistenci výsledných výstupů.

ZÁVĚR

Cílem této bakalářské práce bylo navrhnout, implementovat a ověřit aplikaci určenou pro evidenci pracovní doby. Navržené řešení bylo vytvořeno s důrazem na přehlednost, srozumitelnost, spolehlivost a omezení chybovosti při zpracování údajů. Výsledkem práce je desktopová aplikace, která umožňuje práci se záznamy pracovní doby, jejich úpravu, import vstupních dat a generování výsledných výstupů ve formátu PDF.

V teoretické části byly vymezeny základní principy evidence pracovní doby, analyzovány současné přístupy k jejímu vedení a formulovány požadavky na vhodné softwarové řešení. Následně byly popsány architektonické a technologické prostředky použité při realizaci aplikace. Tato část vytvořila teoretický základ pro návrh praktického řešení.

Praktická část práce se zaměřila na samotný návrh a implementaci aplikace. Byla popsána struktura systému, databázový model, uživatelské rozhraní i algoritmy, které zajišťují klíčové operace aplikace. Zvláštní pozornost byla věnována zejména algoritmu vyrovnávání hodin a algoritmu importu rozvrhu ze systému STAG, protože právě tyto části představují základní logiku celého řešení.

Součástí práce bylo rovněž testování aplikace. Testovací scénáře byly zaměřeny na běžné i hraniční situace, včetně importu dat, vyrovnávání pracovní doby, generování PDF výstupů a práce s uživatelským rozhraním. Výsledky testování ukázaly, že aplikace splňuje stanovené požadavky, správně zpracovává data a poskytuje stabilní a přehledné výstupy. Současně bylo ověřeno, že aplikace vykazuje srovnatelné chování na platformách Windows i Linux.

Přínosem práce je vytvoření funkčního softwarového řešení, které může usnadnit evidenci pracovní doby a zefektivnit práci s její administrativní částí. Do budoucna by bylo možné aplikaci dále rozšířit například o další možnosti exportu, rozšířené testování nebo napojení na další informační systémy.

POUŽITÁ LITERATURA

1. **MICROSOFT**, 2025. *A tour of the C# language*. In: **learn.microsoft.com** [online]. Microsoft, 2025 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/cscz/dotnet/csharp/tour-of-csharp/overview>
2. **MICROSOFT**, 2025. *.NET – Build modern apps and services* (úvod, multiplatformnost). In: **dotnet.microsoft.com** [online]. Microsoft, 2025 [cit. 2025-08-24]. Dostupné z: <https://dotnet.microsoft.com/en-us/>
3. **AVALONIA UI**, 2025. *The MVVM Pattern*. In: **docs.avaloniaui.net** [online]. Avalonia, 2025 [cit. 2025-08-24]. Dostupné z: <https://docs.avaloniaui.net/docs/concepts/the-mvvmpattern/>
4. **REACTIVEUI COMMUNITY**, 2025. *Getting Started*. In: **reactiveui.net** [online]. ReactiveUI, 2025 [cit. 2025-08-24]. Dostupné z: <https://www.reactiveui.net/docs/gettingstarted/>
5. **SQLITE**, 2024. *SQL As Understood By SQLite*. In: **sqlite.org** [online]. SQLite, 2024 [cit. 2025-08-24]. Dostupné z: <https://www.sqlite.org/lang.html>
6. **QUESTPDF**, 2025. *QuestPDF – C# PDF Generation Library*. In: **questpdf.com** [online]. QuestPDF, 2025 [cit. 2025-08-24]. Dostupné z: <https://www.questpdf.com/>
7. **ELECTRON**, 2025. *What is Electron?* In: **electronjs.org/docs/latest** [online]. Electron, 2025 [cit. 2025-08-24]. Dostupné z: <https://electronjs.org/docs/latest/>
8. **PYTHON SOFTWARE FOUNDATION**, 2025. *Python 3.13 documentation*. In: **docs.python.org** [online]. PSF, 2025 [cit. 2025-08-24]. Dostupné z: <https://docs.python.org/>
9. **THE QT COMPANY**, 2025. *Qt for Python (PySide6) – Official documentation*. In: **doc.qt.io** [online]. Qt, 2025 [cit. 2025-08-24]. Dostupné z: <https://doc.qt.io/qtforpython-6/>
10. **JETBRAINS**, 2025. *Compose Multiplatform – Get Started*. In: **jetbrains.com** [online]. JetBrains, 2025 [cit. 2025-08-24]. Dostupné z: <https://www.jetbrains.com/help/kotlinmultiplatform-dev/compose-multiplatform.html>
11. **MICROSOFT**, 2023. *ASP.NET MVC Overview – Model–View–Controller*. In: **learn.microsoft.com** [online]. Microsoft, 2023 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/en-us/aspnet/mvc/overview/older-versions-1/overview/aspnet-mvc-overview>
12. **REACTIVEUI COMMUNITY**, 2025. *The Zen of the ViewModel*. In: **reactiveui.net** [online]. ReactiveUI, 2025 [cit. 2025-08-24]. Dostupné z:

- <https://www.reactiveui.net/docs/handbook/view-models/>
13. **CMU/SEI**, 2010. *What Is Your Definition of Software Architecture?* In: **sei.cmu.edu** [online]. Carnegie Mellon University, 2010 [cit. 2025-08-24]. Dostupné z: <https://www.sei.cmu.edu/library/what-is-your-definition-of-software-architecture/>
 14. **MICROSOFT**, 2023. *Data binding a chování v Avalonia a WPF (Behaviors)*. In: **learn.microsoft.com** [online]. Microsoft, 2023 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/cs-cz/dotnet/desktop/wpf/advanced/behaviors-overview>
 15. **AVALONIA XAML BEHAVIORS**, 2024. *GitHub – Avalonia.Xaml.Behaviors*. In: **github.com/AvaloniaXAMLBehaviors** [online]. Avalonia Team, 2024 [cit. 2025-08-24]. Dostupné z: <https://github.com/AvaloniaXAMLBehaviors>
 16. **AVALONIA COMMUNITY**, 2024. *Behaviors v Avalonia UI*. In: **docs.avaloniaui.net** [online]. Avalonia Community, 2024 [cit. 2025-08-24]. Dostupné z: <https://docs.avaloniaui.net/docs/controls/behaviors>
 17. **MICROSOFT**, 2023. *Common web application architectures – monolitická aplikace*. In: **learn.microsoft.com** [online]. Microsoft, 2023 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/en-us/dotnet/architecture/modern-web-apps-azure/commonweb-application-architectures>
 18. **MICROSOFT**, 2025. *Microservices architecture style – přehled a trade-offy*. In: **learn.microsoft.com** [online]. Microsoft, 2025 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/en-us/azure/architecture/guide/architecturestyles/microservices>
 19. **FOWLER, Martin**, 2015. *Monolith First*. In: **martinfowler.com** [online]. 2015 [cit. 2025-08-24]. Dostupné z: <https://martinfowler.com/bliki/MonolithFirst.html>
 20. **AZURE ARCHITECTURE CENTER**, 2025. *Architecture styles – přehled*. In: **learn.microsoft.com** [online]. Microsoft, 2025 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/cs-cz/azure/architecture/guide/architecture-styles/>
 21. **MICROSOFT**, 2024. *Entity Framework Core – SQLite provider*. In: **learn.microsoft.com** [online]. Microsoft, 2024 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/cscz/ef/core/providers/sqlite/>
 22. **ARTIFEX SOFTWARE**, 2025. *Ghostscript – official documentation*. In: **ghostscript.com** [online]. Artifex Software, 2025 [cit. 2025-08-24]. Dostupné z: <https://www.ghostscript.com/>
 23. **HABJAN, John a kol.**, 2025. *Ghostscript.NET – .NET wrapper pro Ghostscript*. In: **github.com/jhabjan/Ghostscript.NET** [online]. 2025 [cit. 2025-08-24]. Dostupné z: <https://github.com/jhabjan/Ghostscript.NET>

24. **MICROSOFT**, 2025. *Visual Studio – Product documentation (Overview, IDE features)*. In: **learn.microsoft.com** [online]. Microsoft, 2025 [cit. 2025-08-24]. Dostupné z: <https://learn.microsoft.com/cs-cz/visualstudio/ide/?view=vs-2022>
25. **SQLTESTUDIO**, 2025. *SQLiteStudio – Official website (Features, Downloads, Docs)*. In: **sqlitestudio.pl** [online]. 2025 [cit. 2025-08-24]. Dostupné z: <https://sqlitestudio.pl/>

SEZNAM PŘÍLOH

Příloha A: Zdrojový kód aplikace TeacherScheduleApp

PŘÍLOHA A: Zdrojový kód aplikace TeacherScheduleApp

Tato příloha obsahuje zip archiv s projektem z Visual Studia 2022 jménem TeacherScheduleApp